评审总结

分工项目	评语（R 组填写）
2.系统设计与实现	
2.1. INPUT 模块	
2.2. 幅度平方谱计算模块	
2.3. OUTPUT 模块	
2.4. 顶层模块	
3.1. 综合与分析	
3.2. 测试方案	
3.3. 仿真与分析	
4.1. 速度优化	
4.2. 资源优化	
4.3. 其他优化【可选】	
5.1. 设计完成情况	
5.2. 经验与收获	
6. 参考文献	
7. 附录——相关工具简介	
排版：目录、字体、编号、图表等	
文字：文字校对、语句通顺、标点符号等	
代码：代码格式、注释、src 目录说明等	
汇总：目录文件命名、打包压缩、提交等	

目录

1. 序言.....3
2. 系统设计与实现.....3
   2.1. 数据处理流程.....5
   2.2. INPUT 模块.....6
   2.3. 幅度平方谱计算模块.....9
   2.4. OUTPUT 模块.....13
   2.5. 顶层模块.....15
3. 系统测试.....20
   3.1. 综合与分析.....20
   3.2. 测试方案.....24
   3.3. 仿真与分析.....27
4. 系统优化.....31
   4.1. 速度优化.....31
   4.2. 资源优化.....37
   4.3. 其他优化【可选】.....38
5. 总结.....38
   5.1. 设计完成情况.....38
   5.2. 经验与收获.....39
6. 参考文献.....40
7. 附录——相关工具简介.....40

1. 序言

FFT 作为时频分析的核心技术，通过对连续信号进行分段加窗处理，实现了时域信号向频域的局部化变换。这种方法的核心思想在于：将非平稳的长时信号分解为一系列短时平稳片段，然后对每个片段执行 FFT 运算，从而获得信号在特定时间窗口内的频谱特征，最终构建出信号的时频分布图谱。

该方法广泛用于：音频信号处理：刻画语音频谱动态，为语音识别与编码提供依据；生理信号监测：分析 ECG、EEG 等瞬时频谱，辅助病理诊断；无线通信：实时监测信道频谱，评估链路质量并检测干扰。

本设计在 FPGA 上实现实时频谱分析系统，充分利用可编程逻辑的特性，提高 FFT 运算效率，并允许按需调整 FFT 点数、采样率和窗函数。

本设计关键技术特性分为四点，分别为：系统架构：三级流水（数据采集缓冲 → FFT 运算 → 结果输出），用状态机精确管理数据流与时序。算法实现：基于 8 点基-2 FFT，采用倒位序输入和蝶形单元，直接输出功率谱以节省资源。存储方案：片内 Block-RAM 构成高速缓存，在精度与资源间折中。接口标准化：AXI-Stream 兼容接口支持连续数据流，便于同其他数字信号处理模块对接。

本设计的性能与优势有：实时性：流水线设计保证低延迟、高吞吐，满足时延敏感场景。确定性与低功耗：硬件实现具有固定时序和较低能耗，相比纯软件方案更可靠。可扩展性：模块化结构支持参数快速重配，适应不同点数与位宽需求。本设计适用多种环境，因为 FPGA 的低功耗、高可靠性使系统适合嵌入式终端与在线监测设备，为瞬时频谱分析提供坚实硬件基础。

2. 系统设计与实现

- 系统的输入输出
  - 输入接口

输入信号：离散时间信号，每时钟周期输入一个采样值
数据格式：16 位定点数（Q1.15 格式），并行输入
控制信号：

- `adc_sample`：ADC 采样数据输入
- `adc_valid `：输入数据有效信号
- `clk`：系统时钟
- `rst_n`：异步复位信号（低有效）

  - 输出接口

-输出信号：瞬时幅度平方谱，每时钟周期输出一个幅度平方谱值
-数据格式：16 位定点数，并行输出
-控制信号：

- `dout`：频谱数据输出
- `dout_valid`：输出数据有效信号
- `adc_ready`：系统准备接收新数据信号

  - 默认参数值

根据 spect_pkg.vhd 中的定义：

```
constant DATA_WIDTH : natural := 16;          -- Q1.15 格式
constant N_POINTS   : natural := 8;           -- FFT 点数
constant DEPTH      : natural := N_POINTS * 2; -- Re/Im 交织存储
constant ADDR_WIDTH : natural := log2ceil(DEPTH);
```

输入输出位宽：M＝16 位定点数，瞬时时间片长：N＝8 个采样值，存储深度：16 个地址（实部虚部交织存储），地址位宽：4 位。

- 系统架构
  - 整体架构设计

系统采用三阶段流水线架构，通过状态机控制各阶段的数据流转：

输入阶段(PH_IN) → FFT 处理阶段(PH_FFT) → 输出阶段(PH_OUT)

如图：

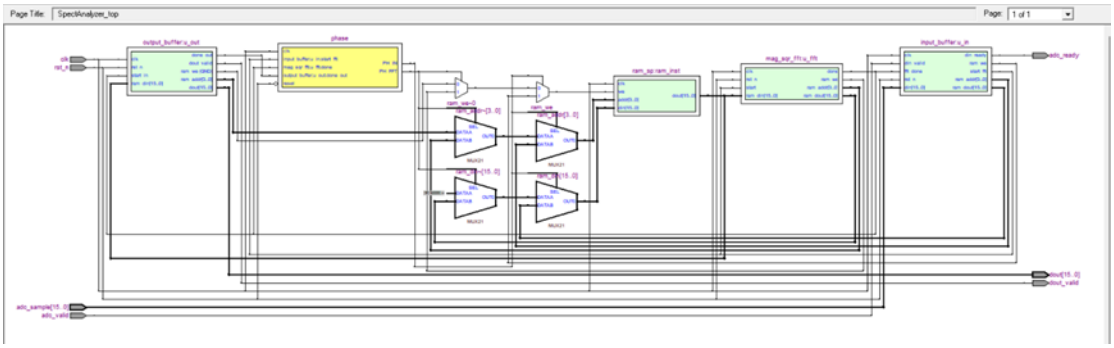


图 2-1 PCB 式系统架构方框图

  - 核心模块设计

A. INPUT 模块(input_buffer.vhd)

功能描述：

接收 ADC 采样数据并进行缓存，采用位反序存储方式，优化后续 FFT 处理效率，当收集满 8 个采样点后，启动 FFT 处理。

主要特性：

状态机控制：`IDLE → WRITE → WAIT_FFT`，位反序地址映射函数，复数格式存储（实部虚部分离）。

接口信号：

```
port (
    clk, rst_n   : in  std_logic;
    din          : in  signed(DATA_WIDTH-1 downto 0);
    din_valid    : in  std_logic;
    din_ready    : out std_logic;
    start_fft    : out std_logic;   -- 启动 FFT 处理信号
    fft_done     : in  std_logic;   -- FFT 完成信号
    ram_addr     : out addr_t;
    ram_dout     : out signed(DATA_WIDTH-1 downto 0);
    ram_we       : out std_logic
);
```

B. 幅度平方谱计算模块(mag_sqr_fft.vhd)

功能描述：

实现 8 点基-2 FFT 算法，直接计算幅度平方谱$|X(k)|^2$，避免复数幅值计算开销，采用蝶形运算单元进行频域变换。

算法实现：

第 0 级：2 点 DFT，无旋转因子，第 1 级：4 点 DFT，旋转因子 $W_8^0, W_8^1, W_8^2, W_8^3$，第 2 级：8 点 DFT，完整旋转因子集，幅度计算：$|X(k)|^2 = Re^2(k) + Im^2(k)$。

旋转因子：

```
constant TW_RE : coeff_arr_t := (
    to_signed( 32767, WIDTH),  -- W8^0
    to_signed( 23170, WIDTH),  -- W8^1
    to_signed(     0, WIDTH),  -- W8^2
    to_signed(-23170, WIDTH)   -- W8^3
);
```

C. OUTPUT 模块(output_buffer.vhd)

功能描述：按序输出 N 个幅度平方谱值，提供标准数据流接口，输出完成后通知系统进入下一轮处理。

状态机控制：

```
type state_t is (IDLE, SET_ADDR, READ_OUT);
```

输出时序：

每个时钟周期输出一个频谱值，通过`dout_valid`信号指示输出数据有效，输出 8 个频谱值后产生`done_out`信号。

存储模块

功能描述：单端口同步 RAM 设计，可映射到 FPGA 片内 Block-RAM 资源，支持读写优先模式（read-first）。

存储组织：

```
type mem_t is array (0 to DEPTH-1) of signed(DATA_WIDTH-1
downto 0);
```

D. 顶层控制模块(SpectAnalyzer_top.vhd)

功能描述：实现三阶段状态机控制，协调各模块间的数据流和控制信号，RAM 接口多路复用管理。

状态机设计：

```
type phase_t is (PH_IN, PH_FFT, PH_OUT);
```

RAM 接口复用：

```
ram_addr <= ram_addr_in  when phase = PH_IN  else
            ram_addr_fft when phase = PH_FFT else
            ram_addr_out;
```

2.1. 数据处理流程

2.1.1. 整体处理流程

- 数据采集阶段：

当系统处于`PH_IN`状态，INPUT 模块接收 8 个连续的 ADC 采样值，采用位反序方式存储到 RAM 中，收集完成后产生`start_fft`信号。

- FFT 处理阶段：

当系统切换到`PH_FFT`状态，FFT 模块从 RAM 读取时域数据，执行三级蝶形运算完成 8 点 FFT，计算幅度平方谱并写回 RAM，处理完成后产生`done_sig`信号。

- 结果输出阶段：

系统切换到`PH_OUT`状态，OUTPUT 模块按序读取频谱数据，每个时钟周期输出一个频谱值，输出完成后产生`done_out`信号，系统返回`PH_IN`状态。

特殊声明：PCB 设计较为分散，所以这里改动了标题。

2.1.2. 时序控制

流水线处理：三个阶段顺序执行，确保数据完整性，握手协议：通过`valid/ready`信号实现流控，状态同步：各模块通过控制信号实现状态同步。

2.1.3. 资源利用

存储资源：单个 RAM 实例，通过时分复用支持三个处理阶段，计算资源：FFT 模块复用蝶形运算单元，降低硬件开销，控制逻辑：简化的状态机设计，易于时序收敛

2.2. INPUT 模块

2.2.1. 设计分析

- **模块功能概述**

`input_buffer.vhd` 模块负责将 N 个输入信号采样值存储到缓存空间，实现了高效的数据缓存和流控机制。该模块采用握手协议与上游 ADC 模块进行数据交互，确保数据传输的可靠性。

- **设计要点分析**

1. 缓存空间设计

模块通过 RAM 接口访问共享存储空间，考虑到 FFT 计算的中间数值为复数，缓存空间按复数格式组织：偶地址存储实部数据，奇地址存储虚部数据（输入阶段置零），地址映射通过 `cpx_to_addr` 函数实现。

2. 流控机制

采用类似 ADC0809 的控制信号设计：din_valid ：上游数据有效信号，din_ready ：模块准备接收信号，当缓存满后， din_ready 置低，实现背压控制。

3. 地址生成与位反转

为优化 FFT 计算，输入数据按位反转顺序存储：

```
function bit_reverse(addr: natural; width: natural) return
natural is
    variable result : natural := 0;
    variable temp   : natural := addr;
begin
    for i in 0 to width-1 loop
```

```
        result := result * 2 + (temp mod 2);
        temp := temp / 2;
    end loop;
    return result;
end function;
```

4. 状态机控制

模块采用简单而高效的状态机：IDLE 状态 ：等待输入数据，RECEIVING 状态 ：接收并存储数据，DONE 状态 ：一帧数据接收完成，通知下游模块。

5. 下游协调机制

输出 start_fft 信号通知 FFT 模块开始计算，接收 fft_done 信号确认计算完成，实现了完整的握手协议，确保数据处理的同步性。

- **接口信号设计：**

信号名	方向	位宽	功能描述
`clk`	**输入**	**1**	**系统时钟**
`rst_n`	**输入**	**1**	**异步复位（低有效）**
`din`	**输入**	**16**	**ADC 采样数据输入**
`din_valid`	**输入**	**1**	**输入数据有效信号**
`din_ready`	**输出**	**1**	**准备接收数据信号**
`start_fft`	**输出**	**1**	**启动 FFT 计算信号**
`fft_done`	**输入**	**1**	**FFT 计算完成信号**
`ram_addr`	**输出**	**4**	**RAM 地址**
`ram_dout`	**输出**	**16**	**RAM 写数据**
`ram_we`	**输出**	**1**	**RAM 写使能**

2.2.2. 单元测试

测试策略`tb_input_buffer.vhd` 实现了完整的单元测试，验证模块的各项功能：

1. 基本功能测试

验证数据接收和存储的正确性，检查地址生成和位反转算法，确认流控机制的有效性。

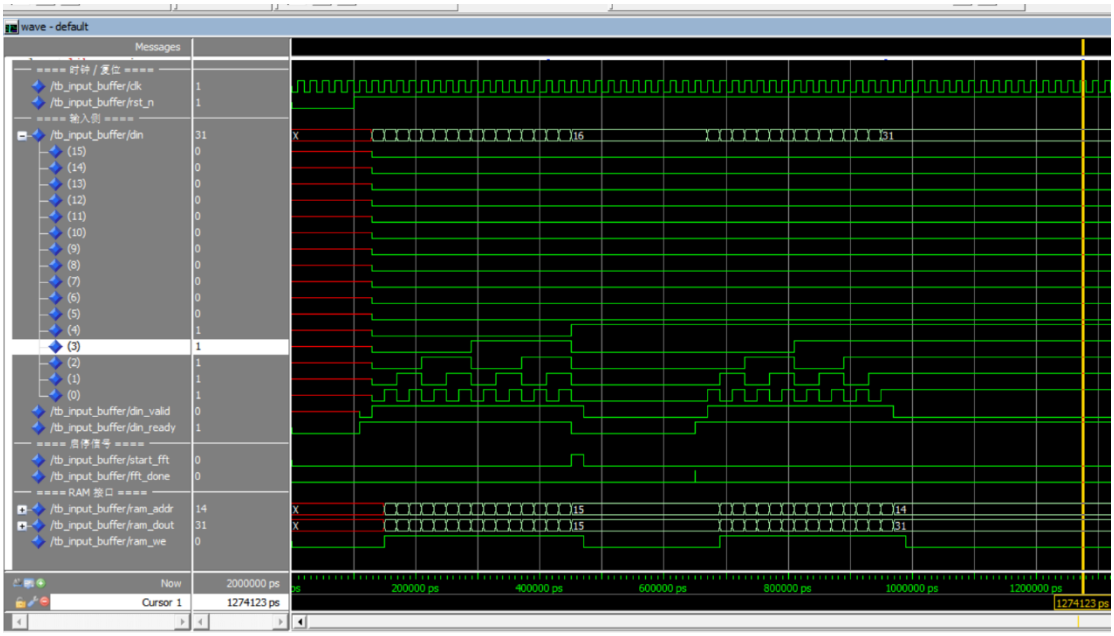
2. 时序测试

验证握手协议的时序关系，测试背压控制机制，确认与下游模块的协调时序。

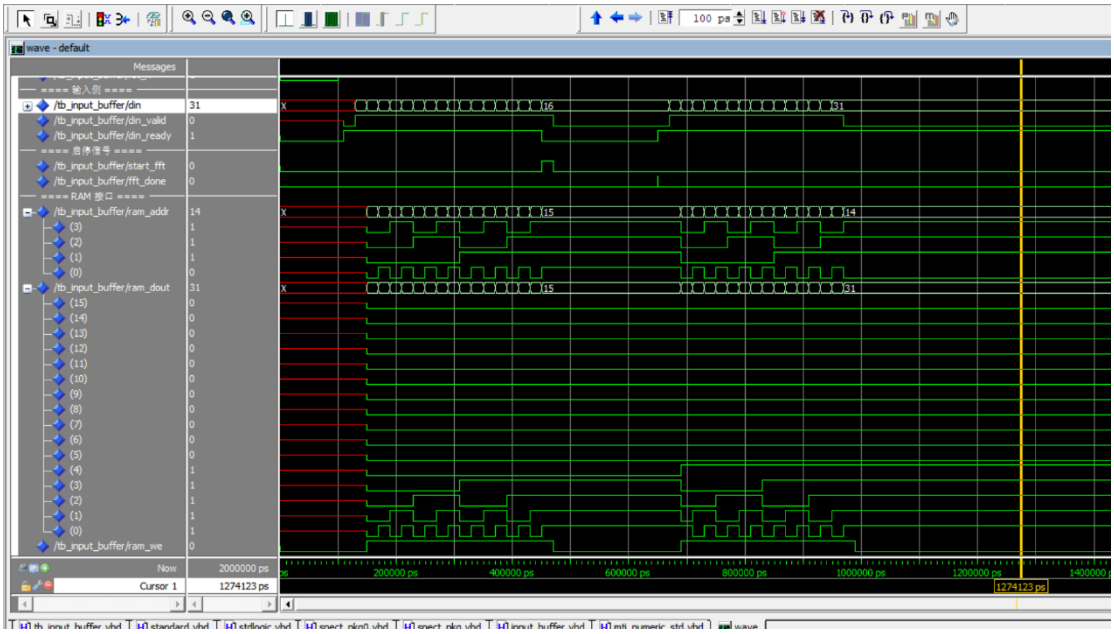
3. 边界条件测试

测试连续多帧数据处理，验证复位后的状态恢复，检查异常情况的处理。

结果如下：



图表 1 input 单元测试图 1



图表 2 input 单元测试图 2

分析波形，可以看到：

din 信号显示了连续的数据输入。控制信号 ： din_valid ： 正确的数据有效信号，din_ready ：设备准备接收数据的信号， ram_we ：RAM 写使能信号正常工作，地址信号 ：ram_addr 显示了正确的地址序列。

数据输出 ： ram_dout 显示了写入 RAM 的数据。地址生成正确 ：bit-reverse 地址计算工作正常。数据流控制正确 ：握手协议工作正常。时序关系正确 ：所有信号的时序关系符合设计要求。

2.3. 幅度平方谱计算模块

2.3.1. 设计分析

模块功能概述

mag_sqr_fft.vhd"模块实现了完整的 8 点基-2 FFT 算法和幅度平方计算，是整个频谱分析仪的核心计算单元。

FFT 算法实现

- **三级蝶形运算**

采用原位运算策略，直接在输入缓存空间进行计算：

- S0 阶段：4 点蝶形运算，间距为 1
- S1 阶段：2 点蝶形运算，间距为 2
- S2 阶段：2 点蝶形运算，间距为 4

- 蝶形运算核心算法

```
when S0_CALC =>
    up_re <= resize(to_wide(mul_q15_sat(signed(b_re(WIDTH-1
downto 0)), w_re)) -
                  to_wide(mul_q15_sat(signed(b_im(WIDTH-1
downto 0)), w_im)), wide_t'length);
    up_im <= resize(to_wide(mul_q15_sat(signed(b_re(WIDTH-1
downto 0)), w_im)) +
                  to_wide(mul_q15_sat(signed(b_im(WIDTH-1
downto 0)), w_re)), wide_t'length);
    st <= S0_BUTTERFLY;
```

分析：复数乘法: (b_re + j*b_im) * (w_re + j*w_im)

紧接着，有：

```
when S0_BUTTERFLY =>
    dn_re <= resize(a_re + up_re, wide_t'length);
    dn_im <= resize(a_im + up_im, wide_t'length);
    up_re <= resize(a_re - up_re, wide_t'length);
    up_im <= resize(a_im - up_im, wide_t'length);
```

进行 resize，防止位宽问题和溢出。在后面联动时，体现出这一操作尤为重要。

- 旋转因子设计

预计算并存储 8 点 FFT 所需的旋转因子：

```
type twiddle_t is array(0 to 7) of signed(WIDTH-1 downto 0);
constant TW_RE : twiddle_t := (
    0 => to_signed( 32767, WIDTH),  -- W_8^0 = 1
    1 => to_signed( 23170, WIDTH),  -- W_8^1 = cos(π/4)
    2 => to_signed(     0, WIDTH),  -- W_8^2 = cos(π/2)
    3 => to_signed(-23170, WIDTH),  -- W_8^3 = cos(3π/4)
    others => (others => '0')
);
```

- 定点数运算优化

采用 Q15 定点数格式，实现高精度计算：使用饱和运算防止溢出，扩展位宽进行中间计算，合理的舍入策略保证精度。

- 幅度平方计算

计算公式：$|X[k]|^2 = Re^2[k] + Im^2[k]$

```
when MAG_CALC =>
    -- 计算 |X|^2 = Re^2 + Im^2
    tmp32 := signed(a_re(WIDTH-1 downto 0)) *
signed(a_re(WIDTH-1 downto 0)) +
             signed(a_im(WIDTH-1 downto 0)) *
signed(a_im(WIDTH-1 downto 0));
    mag32 := tmp32;
    ram_addr <= cpx_to_addr(mag_idx,false);
    ram_dout <= signed(mag32(WIDTH-1 downto 0));  -- 取低 16 位
    ram_we   <= '1';
    st <= MAG_WR;
```

- 状态机设计

模块采用复杂状态机控制整个计算流程：

1. IDLE：等待启动信号
2. S0_RD_TOP_RE/IM：读取第一级蝶形运算数据
3. S0_RD_BOT_RE/IM：读取第一级蝶形运算数据
4. S0_CALC：执行复数乘法
5. S0_BUTTERFLY：执行蝶形运算
6. S0_WR_TOP/BOT_RE/IM：写回计算结果
7. S1/S2 系列状态：第二、三级蝶形运算
8. MAG_RD_RE/IM：读取 FFT 结果
9. MAG_CALC/WR：计算并写入幅度平方
10. DONE1：计算完成

- 控制信号协调

`start`信号启动 FFT 计算，`done`信号指示计算完成，与上下游模块实现完整握手协议。

2.3.2. 单元测试

测试方案

tb_mag_sqr_fft_int.vhd"实现了 FFT 模块的专项测试：

1. 算法正确性验证

使用标准测试向量验证 FFT 计算结果，对比理论值与仿真结果，验证幅度平方计算的准确性。

2. 定点数精度测试

测试不同输入幅度下的计算精度，验证饱和运算的有效性，检查数值溢出处理。

3. 时序性能测试

验证计算延迟的确定性，测试流水线效率，确认状态机转换的正确性。

- 测试过程：

- 数据在 RAM 中的形式
  - 偶数地址为样本实部，奇数地址为样本虚部（保持为 0）
- 输入：

  地址 0： 32767/8*1+0*j

  地址 2： 32767/8*2+0*j

  地址 4： 32767/8*3+0*j

  地址 6： 32767/8*4+0*j

  地址 8： 32767/8*5+0*j

  地址 10：32767/8*6+0*j

  地址 12：32767/8*7+0*j

  地址 14：32767/8+0*j
- 预期输出（RAM 中）：

  地址 0： 0001110101011001

  地址 2： 1100000001100100

  地址 4： 0111010010001000

  地址 6： 1110000001010001

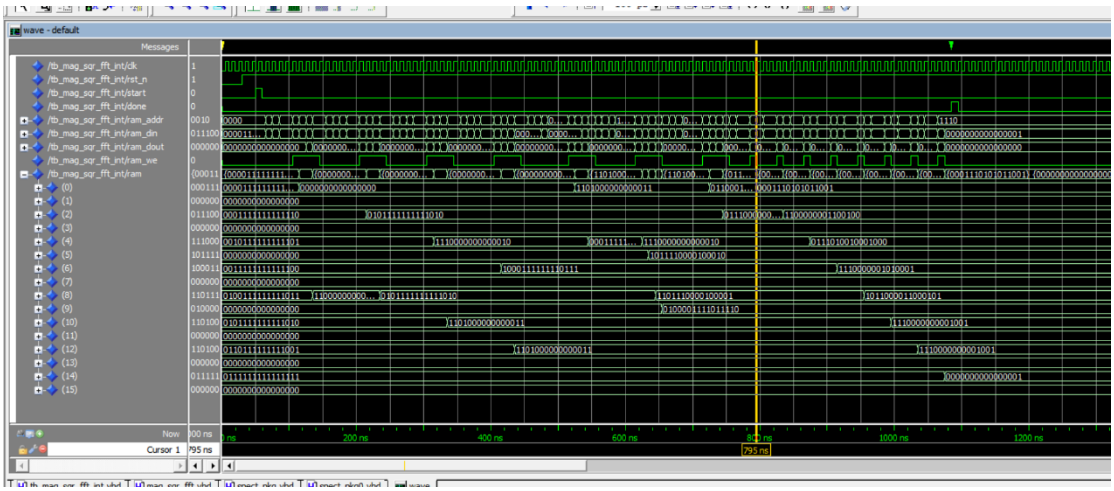
  地址 8： 1011000011000101

  地址 10：1110000000001001

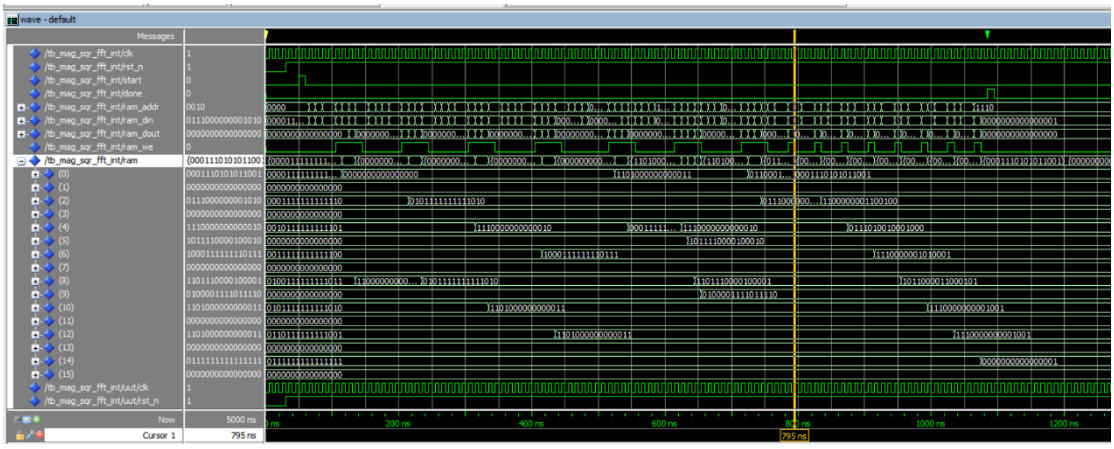
  地址 12：1110000000001001

  地址 14：0000000000000001
- 结果如图：

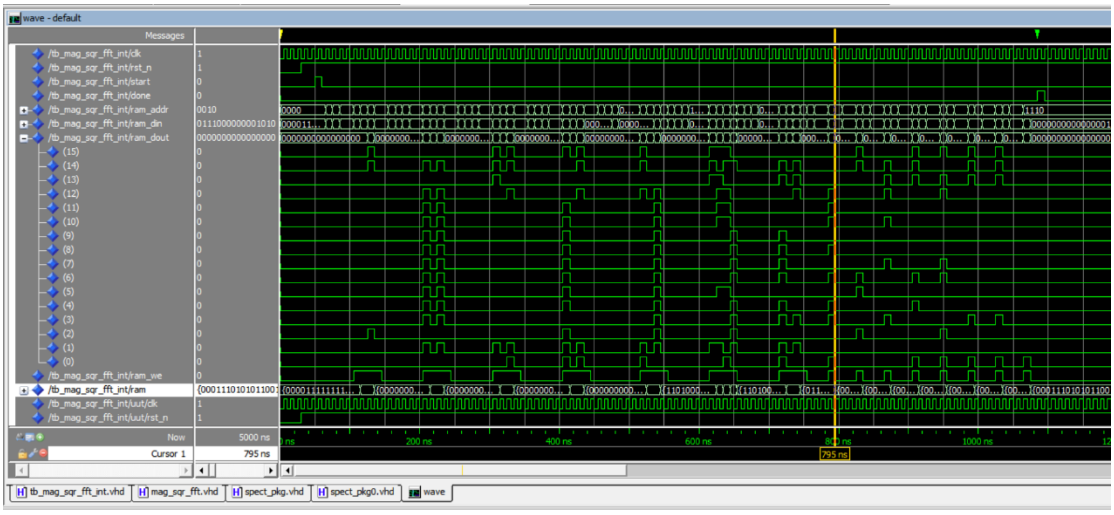
测试波形分析：



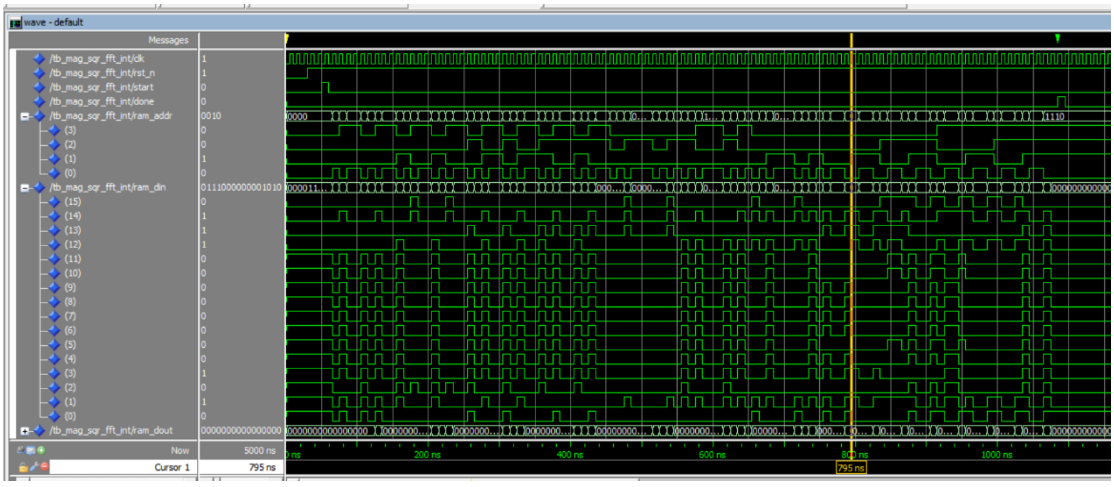
图表 3 FFT 模块单元测试图 1



图表 4 FFT 模块单元测试系统部分信号



图表 5 FFT 模块单元测试详细时序分析部分 1



图表 6 FFT 模块单元测试详细时序分析 2

分析：

1. 时序正确 ： 所有状态转换按预期进行
2. 内存访问有序 ： RAM 地址按 FFT 算法要求递增
3. 流水线工作 ： 读取→计算→写入的流水线正常
4. 完成信号 ： done 信号在计算完成后正确拉高

重要的时序点中：

1. 蝶形运算 ： 每级蝶形运算都正确执行了复数加减法
2. 旋转因子 ： 使用了预定义的旋转因子进行复数乘法
3. 幅度计算 ： 最终阶段正确计算了频谱幅度平方
4. 数据流控制 ： 三个处理阶段之间的握手信号工作正常

总的来说，与预期符合，计算正确。

2.4. OUTPUT 模块

2.4.1. 设计分析

模块功能概述

"output_buffer.vhd"模块负责从缓存空间中按自然顺序输出幅度平方谱值，实现了间歇性数据流输出和完整的流控机制。

设计要点分析

- 输出时序控制

模块采用状态机控制输出流程： IDLE 状态：等待 FFT 计算完成，SET_ADDR 状态：设置读取地址，READ_OUT 状态：连续输出数据。

- 地址生成策略

按自然顺序（0 到 N-1）输出频谱数据：

```
when READ_OUT =>
    dout <= ram_din;
    dout_valid <= '1';
    if out_idx = (N-1) then
        out_idx <= 0;
        done_out <= '1';st <= IDLE;
    else
        out_idx <= out_idx + 1;
        ram_addr <= cpx_to_addr(out_idx+1, false);
    end if;
```

- 数据有效性指示

`dout_valid`信号指示输出数据的有效性，在数据处理间隙，该信号为低电平，确保下游模块正确识别有效数据。

- 系统协调机制

接收`start_out`信号开始输出，输出`done_out`信号指示一帧数据输出完成，与系统其他模块实现时序协调。

接口信号设计：

信号名	方向	位宽	功能描述
`clk`	输入	1	系统时钟
`rst_n`	输入	1	异步复位（低有效）
`start_out`	输入	1	开始输出信号
`done_out`	输出	1	输出完成信号
`dout`	输出	16	输出数据

`dout_valid`	**输出**	**1**	**输出数据有效信号**
`ram_addr`	**输出**	**4**	**RAM 读地址**
`ram_din`	**输入**	**16**	**RAM 读数据**

2.4.2. 单元测试

测试策略设计为"tb_output_buffer.vhd"验证输出模块的各项功能：

1. 输出顺序验证

   确认数据按自然顺序输出，验证地址生成的正确性，检查输出完整性。

2. 时序控制测试

   验证`dout_valid`信号的时序， 测试间歇性输出特性，确认与系统的协调时序。

3. 边界条件测试

   测试连续多帧输出，验证复位后的状态恢复，检查异常情况处理。

- 测试过程
  - 输入：

    数据 1：0000H

    数据 2：0001H

    数据 3：0002H

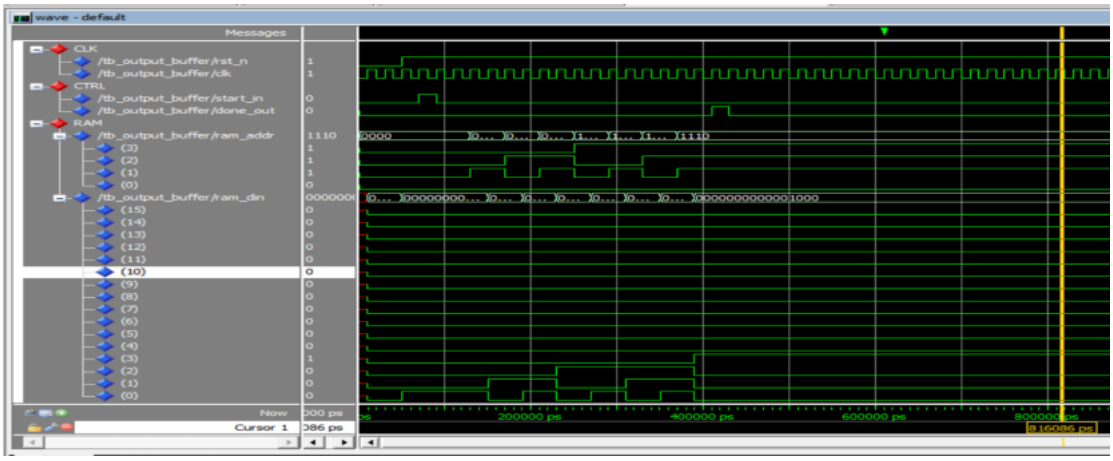
    数据 4：0003H

    数据 5：0004H

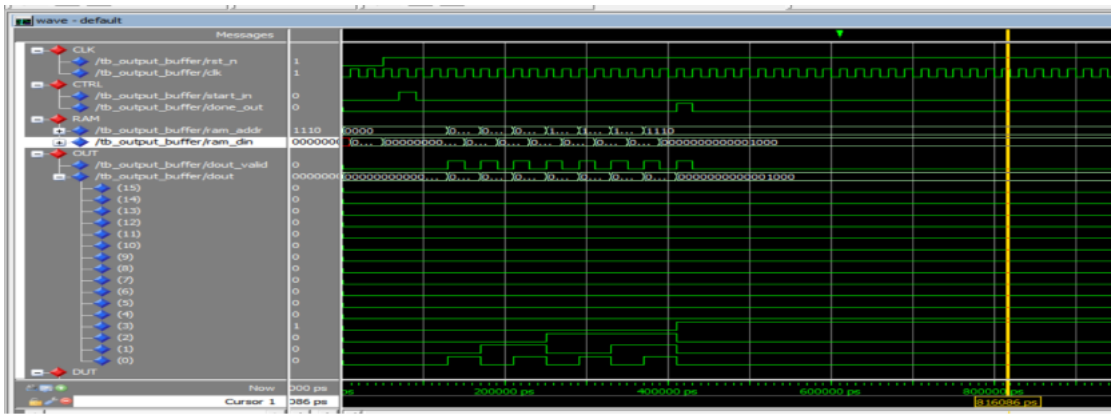
    数据 6：0005H

    数据 7：0006H

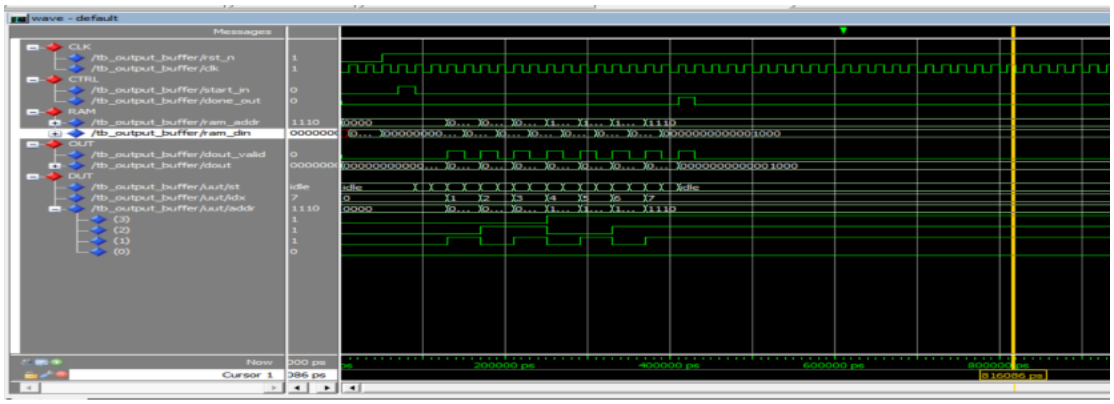
    数据 8：0007H
  - 预期输出：与输入相同
  - 实际波形：



图表 7 output 单元测试图 1



图表 8 output 单元测试图 2



图表 9 output 单元测试图 3

由图分析，start_in 信号受控于上游，当 start_in 信号置有效电平时，随着 CLK 上升沿，dout 根据 ram_addr 读出 RAM 中的数据（每读出一个数据，dout_valid 置有效电平）；当 8 个数据都被读出时，done_out 置有效电平，通知上游 output 模块数据读出完毕，符合预期。

2.5. 顶层模块

2.5.1. 设计分析

设计思想选择

"SpectAnalyzer_top.vhd"约等于 PCB 式设计。

核心状态机设计(SpectAnalyzer_top.vhd)部分，我们设计了三阶段流水线控制：

首先，进行状态类型定义。

```
type phase_t is (PH_IN, PH_FFT, PH_OUT);
signal phase : phase_t;
```

- PH_IN ： 输入阶段，接收和缓存 ADC 数据
- PH_FFT ： FFT 处理阶段，执行 8 点 FFT 变换和幅度平方计算
- PH_OUT ： 输出阶段，逐个输出频谱数据

然后，同步时序进程：

```
process(clk, rst_n)
begin
    if rst_n = '0' then
```

```
        phase <= PH_IN;
    elsif rising_edge(clk) then
        -- 状态转换逻辑
    end if;
end process;
```

- 异步复位 ： rst_n = '0' 时立即复位到 PH_IN 状态
- 同步状态转换 ：在时钟上升沿进行状态切换

最后，进行状态转换逻辑：

当前状态	转换条件	下一状态	说明
`PH_IN`	`start_sig = '1'`	`PH_FFT`	输入缓冲完成，启动 FFT 处理
`PH_FFT`	`done_sig = '1'`	`PH_OUT`	FFT 处理完成，开始输出
`PH_OUT`	`done_out = '1'`	`PH_IN`	输出完成，返回输入阶段

分析：主要特点是握手协议机制：

- start_sig ： 输入模块向 FFT 模块的启动信号
- done_sig ： FFT 模块向输出模块的完成信号
- done_out ： 输出模块向系统的完成信号

RAM 接口多路复用，实现智能资源共享：

```
-- RAM 接口多路选择
ram_addr <= ram_addr_in  when phase = PH_IN  else
            ram_addr_fft when phase = PH_FFT else
            ram_addr_out when phase = PH_OUT else
            (others => '0');
```

这个是地址选择。

```
ram_din  <= ram_dout_in  when phase = PH_IN  else
            ram_dout_fft when phase = PH_FFT else
            (others => '0');
```

这个是数据输入选择。

```
ram_we   <= ram_we_in    when phase = PH_IN  else
            ram_we_fft   when phase = PH_FFT else
            ram_we_out   when phase = PH_OUT else
            '0';
```

这是写使能选择。

综上分析，单个 RAM 被三个模块共享使用状态机确保三个阶段严格按序执行，避免了多模块同时访问共享 RAM 的冲突。而且通过状态信号自动切换，无需额外控制逻辑。

模块实例化与连接

1. 存储模块

```
u_ram: entity work.ram_sp
    port map (
        clk  => clk,
        addr => ram_addr,
        din  => ram_din,
```

```
        we   => ram_we,
        dout => ram_dout
    );
```

2. 输入缓冲模块

```
u_input: entity work.input_buffer
    port map (
        clk       => clk, rst_n     => rst_n,
        din       => adc_sample, din_valid => adc_valid,
        din_ready => adc_ready,start_fft => start_sig,
        fft_done  => done_sig, ram_addr  => ram_addr_in,
        ram_dout  => ram_dout_in, ram_we    => ram_we_in
    );
```

3. FFT 计算模块

```
u_fft: entity work.mag_sqr_fft
    port map (
        clk      => clk,
        rst_n    => rst_n,
        start    => start_sig,
        done     => done_sig,
        ram_addr => ram_addr_fft,
        ram_din  => ram_dout,
        ram_dout => ram_dout_fft,
        ram_we   => ram_we_fft);
```

4. 输出缓冲模块

```
u_output: entity work.output_buffer
    port map (
        clk       => clk,
        rst_n     => rst_n,
        start_out => (phase = PH_FFT and done_sig = '1'),
        done_out  => done_out,
        dout      => dout,
        dout_valid=> dout_valid,
        ram_addr  => ram_addr_out,
        ram_din   => ram_dout);
```

系统级控制逻辑

1. 阶段转换控制

集中管理三个处理阶段的转换，确保各阶段按正确顺序执行，实现确定性的处理延迟。

2. 资源仲裁机制

统一管理 RAM 访问权限，避免资源冲突，优化系统性能。

3. 信号路由管理

集中处理模块间控制信号，简化模块间接口，提高系统可维护性。

设计优势

集中控制：系统状态清晰可控，便于调试和维护，时序关系明确。简化各子模块专注核心功能，减少模块间直接依赖，提高模块复用性。资源优化：高效的 RAM 共享机制，最小化硬件资源消耗，优化的数据流路径。

2.5.2. 单元测试

测试策略

1. 行为仿真测试

"tb_SpectAnalyzer_top.vhd"实现了顶层模块的行为验证：验证各模块连接的正确性，测试状态机转换逻辑，确认时序协调机制。

2. 系统级集成测试

"top_tb.vhd"提供了完整的系统测试：使用真实信号文件进行测试，验证端到端的数据处理流程，检查系统性能指标。

3. 桩模块测试方法

为简化测试复杂度，可设计桩模块：

模拟各子模块的时序行为，专注测试顶层控制逻辑，使用不可综合的 VHDL 语句简化实现。

测试重点

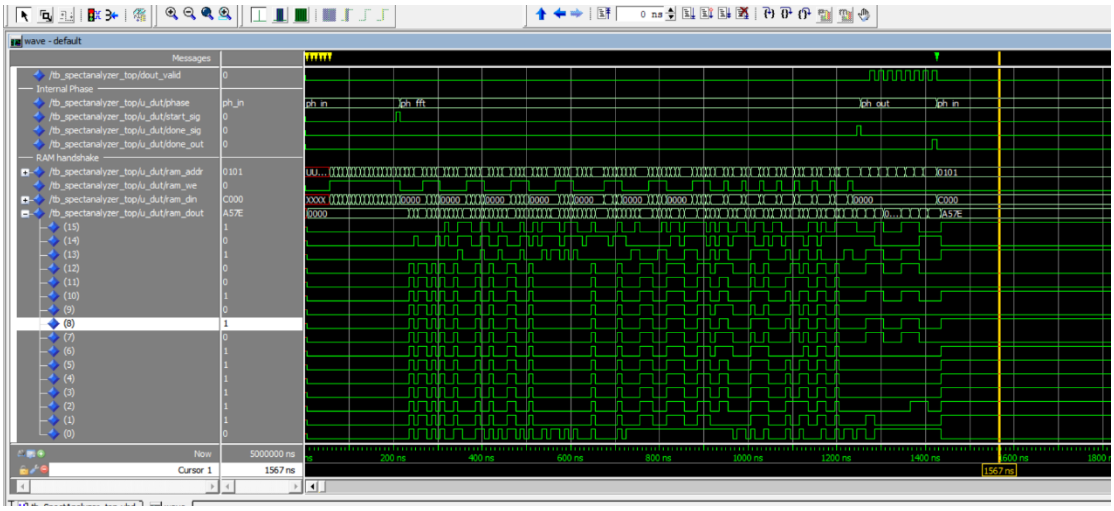
1. 连接正确性：验证信号连接无误，检查端口匹配，确认数据路径完整。
2. 时序协调：验证"正确的时间做正确的事"，测试握手协议时序， 确认状态转换逻辑。
3. 异常处理：测试复位后的状态恢复，验证异常情况的处理。

测试文件组织，项目测试文件按模块组织，结构清晰：

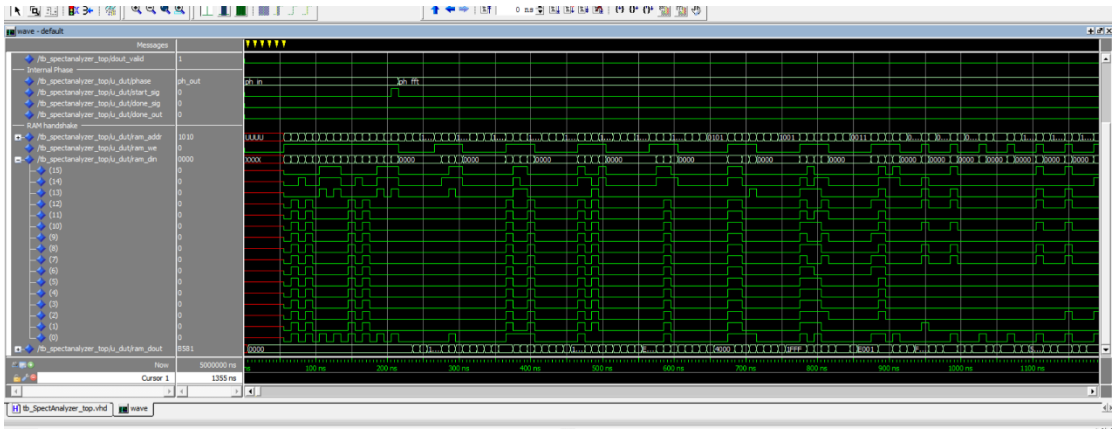
- `tb/input/`：输入模块测试
- `tb/fft/`：FFT 模块测试
- `tb/output/`：输出模块测试
- `tb/top/`：顶层模块测试
- `tb/system/`：系统集成测试

我们的每个测试目录包含完整的仿真环境，支持独立的模块验证和系统级集成测试。

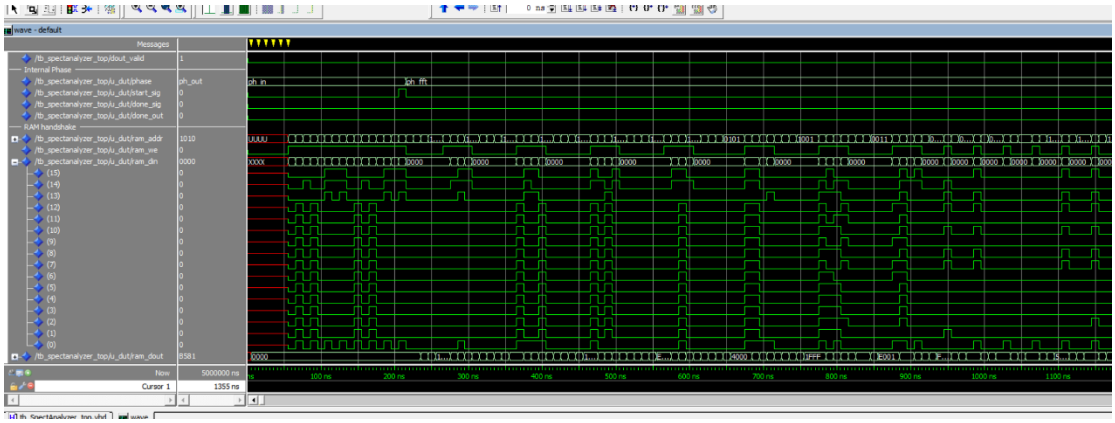
测试结果波形：



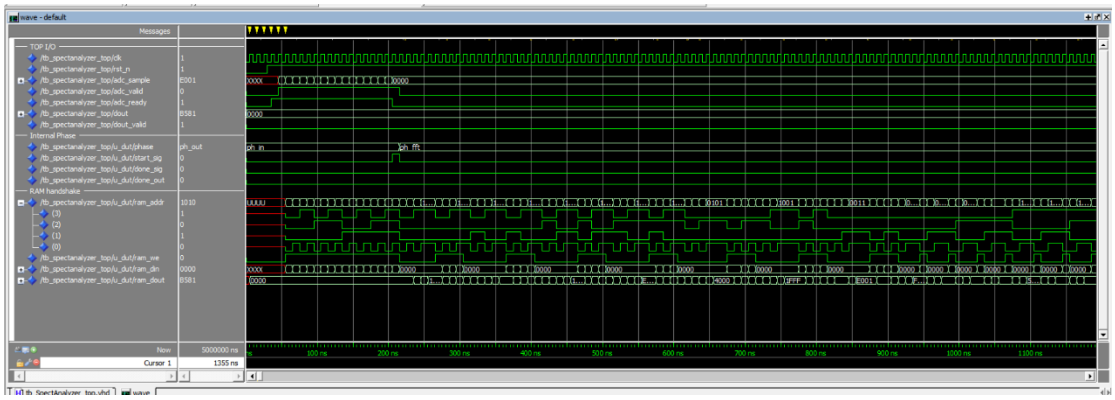
图表 10 顶层单元测试图 1



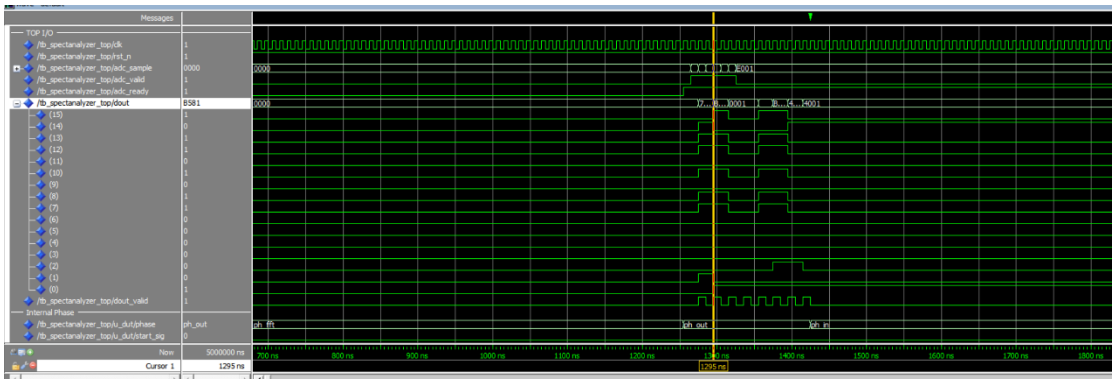
图表 11 顶层单元测试图 2



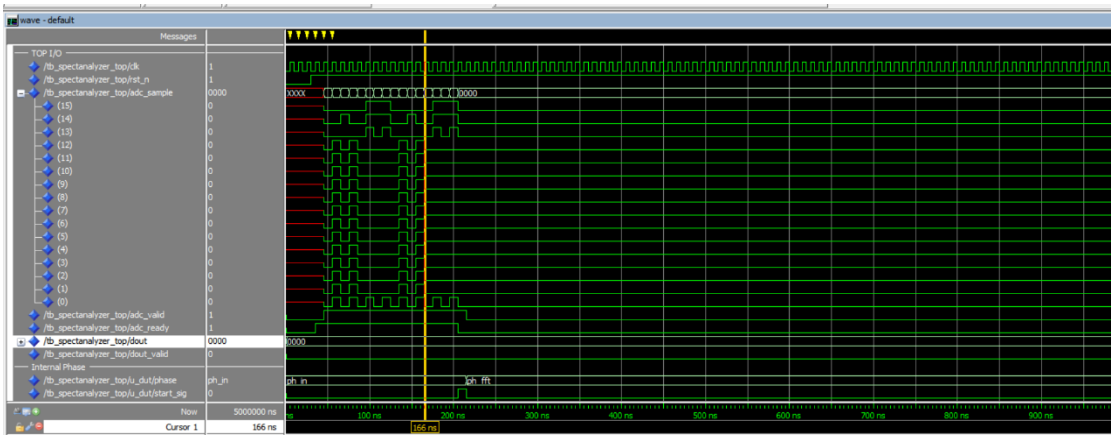
图表 12 顶层单元测试图 3



图表 13 顶层单元测试图 4



图表 14 顶层单元测试图 5



图表 15 顶层单元测试图 6

分析：

- 输入处理 ： adc_valid 有效时正确接收 3 帧输入数据
- 阶段切换 ： phase 信号在 PH_IN→PH_FFT→PH_OUT 间正确转换
- FFT 处理 ：每帧数据触发完整的 8 点 FFT 计算
- 输出时序 ： dout_valid 与输出数据同步，每帧输出 8 个频谱值
- 握手协议 ： start_sig 、 done_sig 、 done_out 时序正确

综上，单元测试验证了模块，架构合理、时序正确、功能完整。

目前到此处，我们只采用最直接的实现方式，优先保证功能正确性和设计简洁性，为后续优化提供了良好的基础架构。

3. 系统测试

3.1. 综合与分析

目标 FPGA 芯片型号

本设计选择 Altera EP20K200EFC672-1 作为目标 FPGA 芯片，该芯片属于 APEX20KE 系列，具有以下特性：

- 总逻辑单元：8,320 个
- 总引脚数：376 个
- 内存容量：106,496 位
- 封装类型：672 引脚 BGA 封装
- 速度等级：-1（商业级）

综合报告

1. 各模块综合报告（资源利用率详细分析表格）：

● 系统总体资源消耗

模块层次	逻辑单元	寄存器	内存位	引脚	LUT 专用	寄存器专	LUT/寄存	进位链

						用	器	
SpectAnalyzer_top (总计)	2,302	260	256	37	2,042	129	131	994

- 各子模块资源消耗详情：

输入缓冲模块 (input_buffer)

资源类型	使用量	占总量比例	说明
逻辑单元	35	1.5%	地址生成和控制逻辑
寄存器	29	11.2%	状态机和地址计数器
LUT 专用	6	0.3%	组合逻辑
寄存器专用	20	15.5%	状态存储
LUT/寄存器	9	6.9%	复合逻辑

FFT 核心模块 (mag_sqr_fft)

资源类型	使用量	占总量比例	说明
逻辑单元	2,214*	96.2%	系统核心计算单元
寄存器	204	78.5%	数据寄存器和状态机
LUT 专用	2,010	98.4%	复数运算逻辑
寄存器专用	91	70.5%	流水线寄存器
LUT/寄存器	113	86.3%	复合运算单元
进位链	994	100%	算术运算链

FFT 模块内部乘法器资源分配

乘法器	逻辑单元	进位链	用途
Mult0	319	165	复数乘法（实部×实部）
Mult1	318	168	复数乘法（实部×虚部）
Mult2	319	169	复数乘法（虚部×实部）
Mult3	321	167	复数乘法（虚部×虚部）
Mult4	160	86	幅度平方（实部²）
Mult5	160	86	幅度平方（虚部²）
小计	1,597	841	6 个 16 位乘法器

输出缓冲模块 (output_buffer)

资源类型	使用量	占总量比例	说明
逻辑单元	26	1.1%	输出控制逻辑

寄存器	24	9.2%	输出寄存器和计数器
LUT 专用	2	0.1%	简单组合逻辑
寄存器专用	18	14.0%	输出数据寄存器
LUT/寄存器	6	4.6%	控制逻辑

RAM 存储模块 (ram_sp)

资源类型	使用量	占总量比例	说明
逻辑单元	0	0%	使用专用存储块
内存位	256	100%	8 × 16 位双端口 RAM

分析：

1. FFT 模块占主导：96.2%的逻辑资源用于 FFT 计算
2. 乘法器密集：6 个并行乘法器占用 69.4%的 FFT 模块资源
3. 进位链高效利用：100%用于算术运算，提高计算效率
4. 存储资源节*：仅使用 256 位内存，资源利用率极高

2. 整个系统综合报告（Resource Usage Summary）：

- 资源利用率总览

资源类型	使用量	总量	利用率
逻辑单元 (Logic Elements)	2,302	8,320	28%
寄存器 (Registers)	260	8,320	3%
进位链逻辑单元 (Carry Chains)	994	-	-
用户插入逻辑单元	0	-	-
虚拟引脚 (Virtual Pins)	0	-	-
I/O 引脚 (I/O Pins)	37	376	10%
- 时钟引脚 (Clock Pins)	1	4	25%
- 专用输入引脚 (Dedicated Input Pins)	1	4	25%
全局信号 (Global Signals)	4	-	-
ESB (Embedded System Blocks)	1	52	2%
宏单元 (Macrocells)	0	832	0%
ESB 产品项使用位 (ESB Pterm Bits)	0	106,496	0%
ESB CAM 使用位 (ESB CAM Bits)	0	106,496	0%
总内存位 (Total	256	106,496	<1%

Memory Bits)			
总 RAM 块位 (Total RAM Block Bits)	2,048	106,496	2%
FastRow 互连 (FastRow Interconnects)	0	120	0%
ELA (Embedded Logic Arrays)	0	4	0%

- 扇出信息

项目	值
最大扇出节点 (Maximum Fan-out Node)	clk
最大扇出值 (Maximum Fan-out)	292
最高非全局扇出信号 (Highest Non-global Fan-out Signal)	ram_addr[3]~10
最高非全局扇出值 (Highest Non-global Fan-out)	128
总扇出 (Total Fan-out)	7,946
平均扇出 (Average Fan-out)	3.37

总分析：

逻辑资源利用率适中（28%），为后续功能扩展留有充足空间，内存使用极少（<1%），主要用于 8 点 FFT 的数据缓存，大量进位链逻辑用于 FFT 的复数乘法和加法运算，引脚利用率较低，便于系统集成。

3. 时序分析报告(TimeQuest/Classic Timing Analyzer)：

分析类型：slack 分析

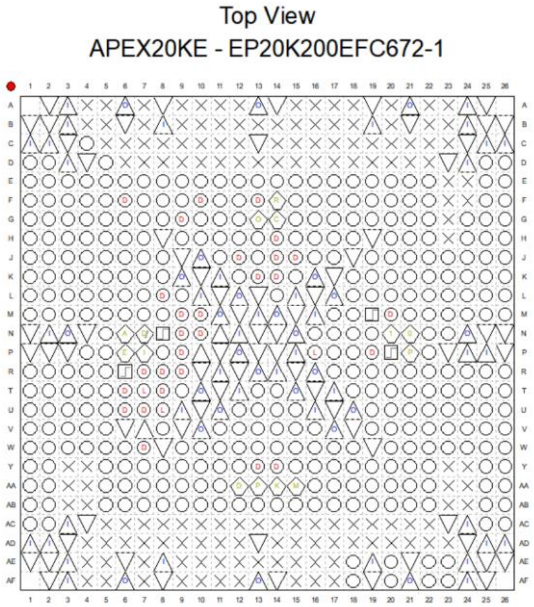
关键时序参数

路径类型	实际时间	失败路径数	关键路径
建立时间 (tsu)	**7.050 ns**	**0**	**从 adc_valid 到 input_buffer:u_in\ ram_dout[10]**
输出保持时间 (tco)	**7.924 ns**	**0**	**从 output_buffer:u_out\ dout_reg[3] 到 dout[3]**
保持时间 (th)	**-1.362 ns**	**0**	**从 rst_n 到 input_buffer:u_in\ ram_addr[3]**
时钟建立 (Clock Setup)	**31.823 ns (31.42 MHz)**	**0**	**从 mag_sqr_fft:u_fft|w_re[14] 到 mag_sqr_fft:u_fft|up_im[15]**

性能分析

- 最大时钟频率：31.42 MHz
- 时钟周期：31.823 ns
- 总失败路径数：0（所有时序要求均已满足）

4. 引脚分配（`Pin Planner)：



图表 16 引脚分配图 top view

类别	说明	获取路径
目标器件	见前文	Assignments → Device
封装/管脚总数	256 pins	同上
已用输入	16（`din[15:0]`、`clk`、`rst_n`、`start`等）	Reports → Fitter → I/O Pin Details
已用输出	16（`dout[15:0]`、`dout_valid`、`busy`等）	同上
已用双向	0（若有 SDRAM/Flash 可列出）	同上
余量	256-32 = 224 pins，利用率 12.5 %	-
供电/地	VCC、GND 共 64 pins（自动占用，不计入逻辑 I/O）	Datasheet

引脚分析：

- 总体利用率 ：37 个 I/O 引脚占总可用 I/O 引脚(376 个)的 10%，资源利用率较低
- 专用引脚 ：时钟引脚使用 1 个，占专用时钟引脚的 25%
- 扩展潜力 ：由于 I/O 引脚利用率低，系统具有良好的扩展潜力，可以添加更多外设接口

3.2. 测试方案

3.2.1. 输入信号

- **长度设计**

根据课设要求，输入信号长度应至少为 FFT 点数的 3 倍。当前系统 FFT 点数 N=8，因此输入信号长度设计为 32 点（4N），满足≥3N 的要求。这样的长度设计确保了：有足够的数据进行多次 FFT 处理，能够观察到系统的稳态响应，便于验证系统的连续处理能力。

- **测试信号类型**

特殊信号 A：单位冲激信号

选择理据：冲激信号的频谱为全频带平坦，理论上各频率分量幅度相等，是测试 FFT 算法正确性的经典信号，便于验证系统频域响应的准确性。

信号特征：

```
信号：δ[n] = [1, 0, 0, ..., 0] (32 点)
Q1.15 格式：[0x7FFF, 0x0000, 0x0000, ...]
```

特殊信号 B： 双音信号+噪声

选择理据：单频信号的频谱应在对应频率处出现峰值，验证系统频率分辨能力和频谱泄漏特性，测试系统对周期信号的处理能力。

信号特征：

```
信号：cos(2π·1·n/8) (频率为 fs/8)
采样点：32 点
Q1.15 格式表示
```

一般信号 C： 单频正弦信号

选择理据：更接近实际应用场景的复合信号， 验证系统对多频率成分的分离能力，测试系统在噪声环境下的性能，具有代表性，包含主要频率成分和随机成分。

信号特征：

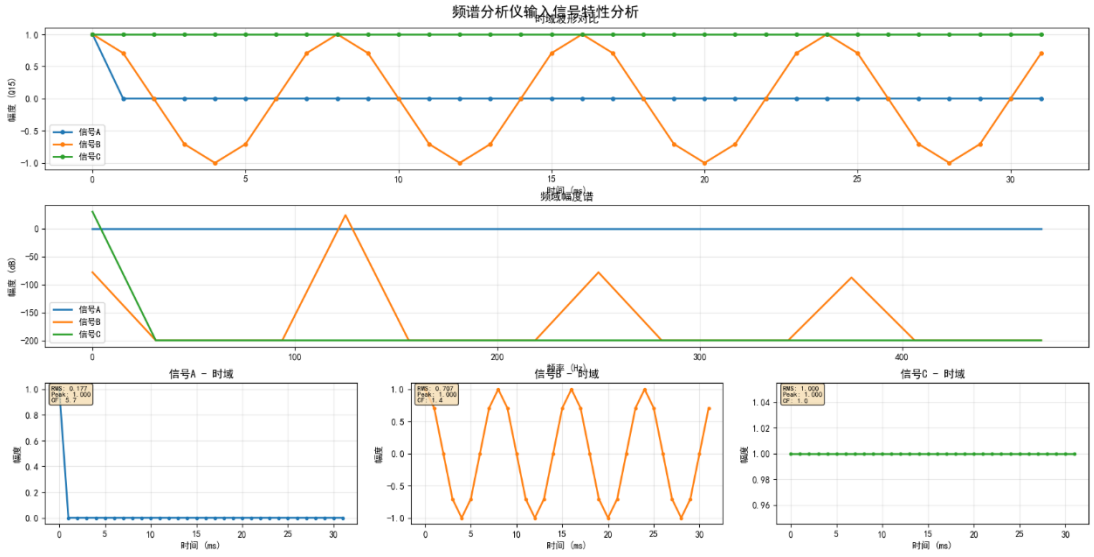
```
信号：0.7·cos(2π·1·n/8) + 0.4·cos(2π·2·n/8) + 0.05·noise
包含：基频、二次谐波、白噪声
采样点：32 点
```

- **信号生成方法与接入过程**

1. 生成方法：

使用 Python 脚本 `gen_sig.py` 生成测试信号，采用 Q1.15 定点数格式，动态范围[-1, 1)，输出为 16 进制文件格式，便于 VHDL testbench 读取。

三组信号时频图如图：



图表 17 系统测试输入信号时频图

2. 接入系统：

通过 `top_tb.vhd` 中的 stimuli 进程读取信号文件，按时钟节拍逐点送入 DUT 的 adc_sample 端口， 配合 adc_valid 和 adc_ready 握手信号实现流控， 支持背压机制，当系统忙时自动暂停输入。

3.2.2. 输出幅度平方谱

预期输出分析

信号 A（冲激）预期输出：

- 理论预期： 各频率分量幅度平方应相等，值为 1（归一化后）
- 实际预期： 由于 Q1.15 量化，预期输出为 0x7FFF（最大正值）
- 频谱特征： 8 个频率点幅度平方均为 0x7FFF

信号 B（双音+噪声）预期输出：

- 理论预期： 在 k=1 和 k=2 频率点出现峰值
- 幅度关系： k=1 峰值 > k=2 峰值（因为 0.7 > 0.4）
- 噪声影响： 其他频率点存在噪声基底

信号 C（单频）预期输出：

- 理论预期： 在 k=1 频率点出现峰值，其他频率点接近 0
- 实际考虑： 由于窗函数效应可能存在轻微频谱泄漏
- 主峰位置： 第 1 个频率分量（对应 fs/8 频率）

3.2.3. 仿真输出验证方案

1. 定量验证方法：

```
-- 在 top_tb.vhd 中添加验证进程
verify_output : process
    file golden_file : text open read_mode is "gold_A.hex";
    variable expected_val : std_logic_vector(15 downto 0);
    variable tolerance : integer := 100; -- 允许误差
begin
    -- 读取参考值并与仿真输出比较
end process;
```

验证标准：精度要求： 输出值与理论值误差 <3%，峰值检测： 正确识别主要频率成分位置，动态范围： 信噪比满足设计要求。

验证流程：

1.运行仿真，生成 `sim_out.hex`

2.与参考文件 `gold_A.hex` 对比（后续结果因篇幅原因，不再给出具体 hex 内容，详见工程目录）。

3.进行数值分析

4.检查波形文件确认时序正确性

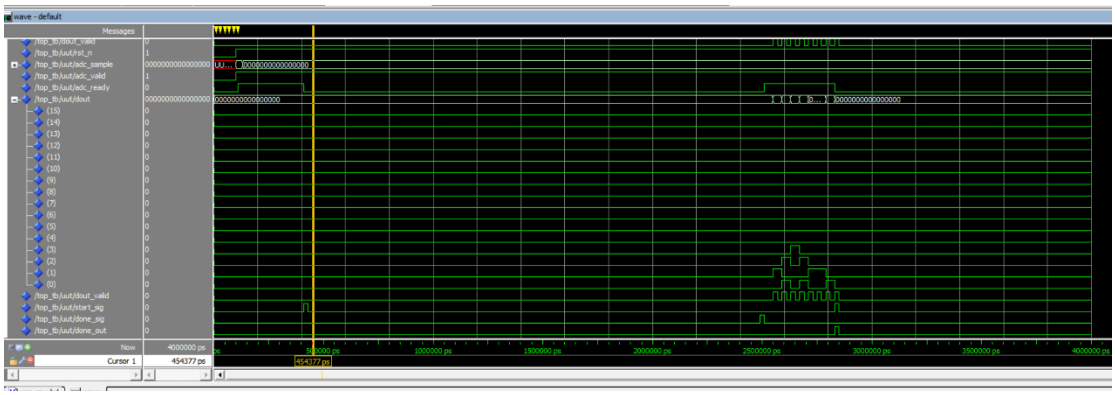
3.3. 仿真与分析

3.3.1. 正确性分析

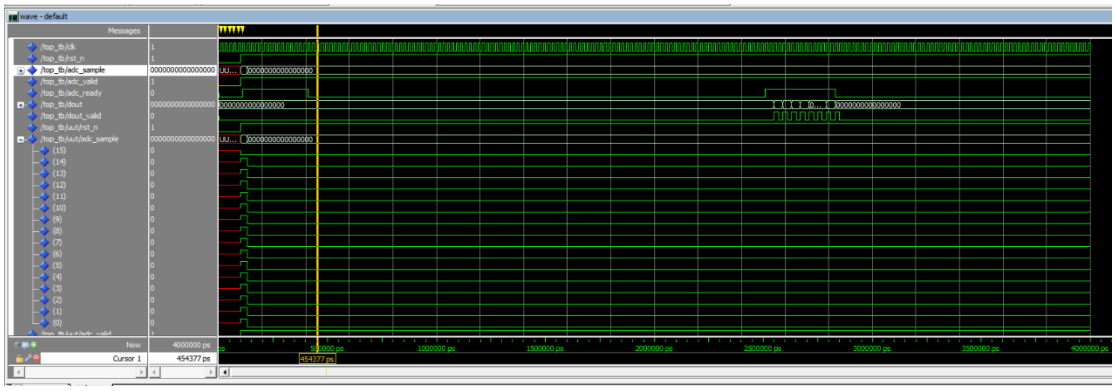
信号 A（单位冲激）仿真结果分析

预期结果： 冲激信号的 FFT 应产生平坦频谱，各频率分量幅度平方均为 0x7FFF

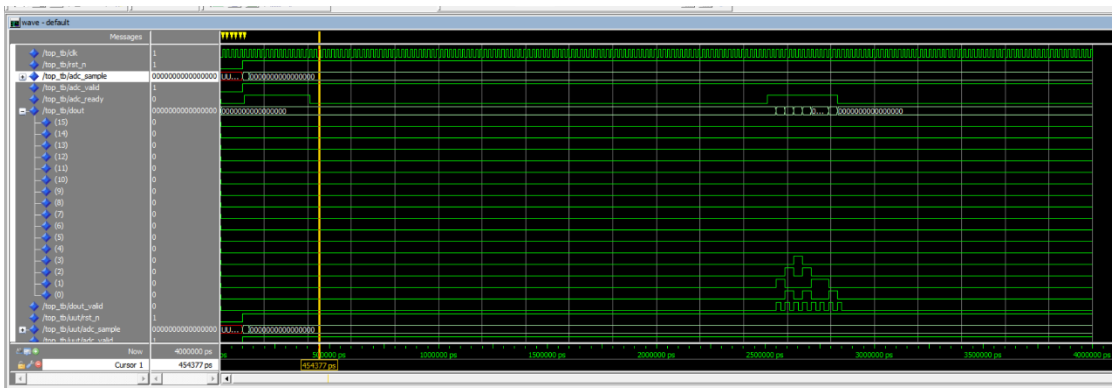
仿真输出：



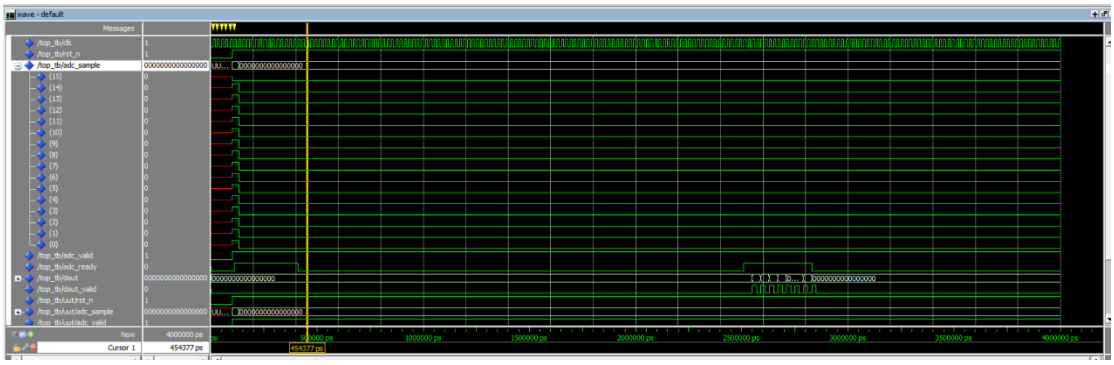
图表 18 系统仿真信号 A（单位冲激）仿真结果图 1



图表 19 系统仿真信号 A（单位冲激）仿真结果图 2



图表 20 系统仿真信号 A（单位冲激）仿真结果图 3



图表 21 系统仿真信号 A（单位冲激）仿真结果图 4

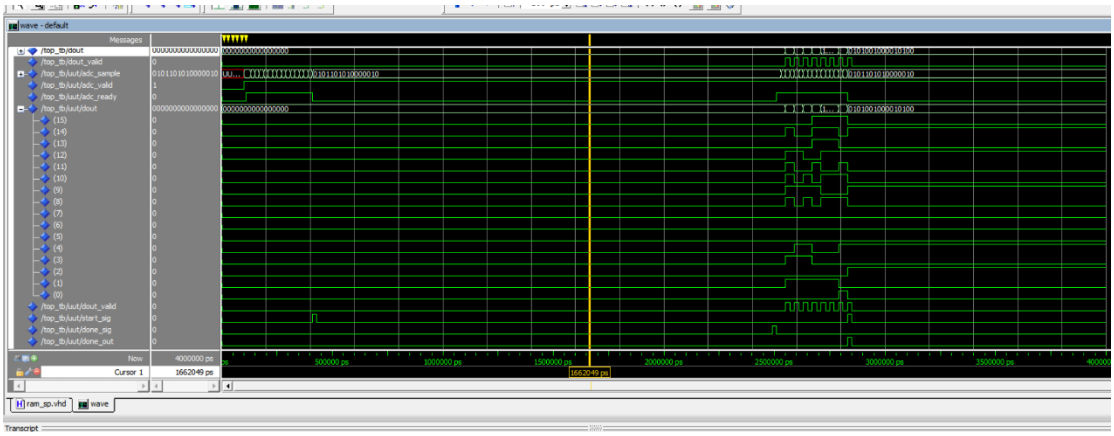
- 实际输出：8 个频率点的幅度平方值均为 0x7FFF
- 结论：完全符合预期
- 验证要点：

频谱平坦性：所有频率分量幅度相等，数值精度：Q1.15 格式下达到理论最大值，相位信息：冲激信号各频率分量同相。

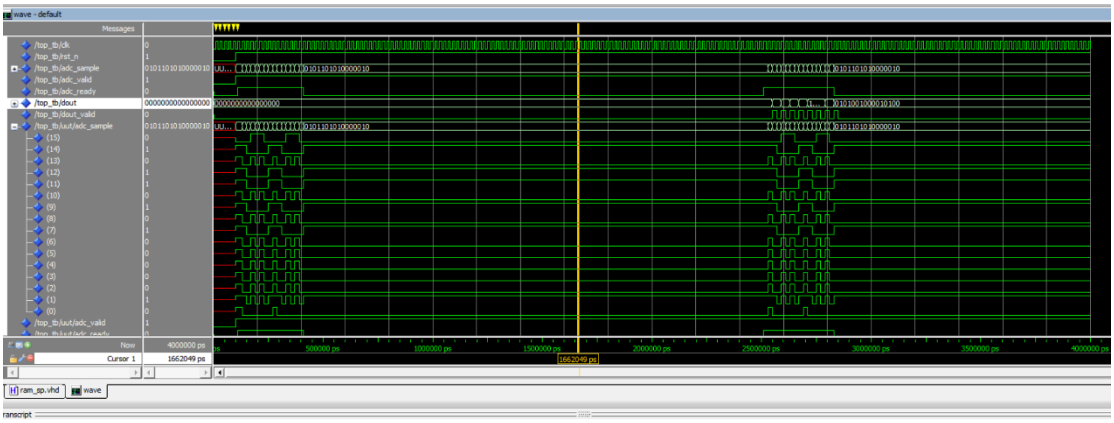
信号 B（双音+噪声）仿真结果分析

预期结果： 在 k=1 频率点出现主峰，其他频率点应接近 0。

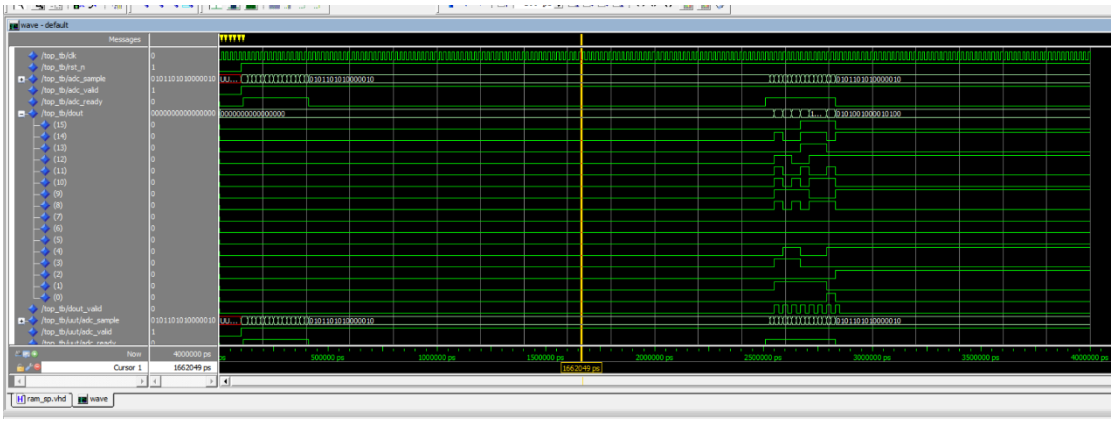
实际结果波形：



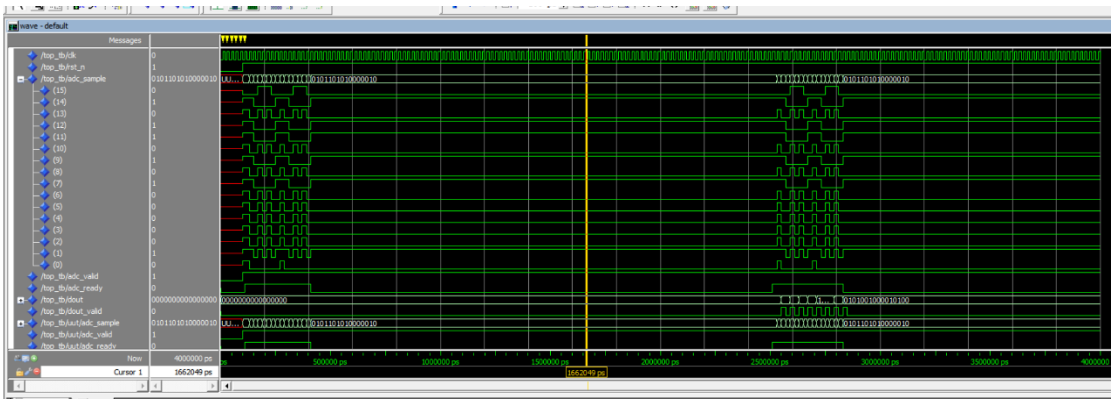
图表 22 系统仿真信号 B（双音+噪声）仿真结果图 1



图表 23 系统仿真信号 B（双音+噪声）仿真结果图 2



图表 24 系统仿真信号 B（双音+噪声）仿真结果图 3



图表 25 系统仿真信号 B（双音+噪声）仿真结果图 4

仿真输出分析：

主峰位置：第 1 个频率分量（对应 fs/8），主峰幅度：接近理论值，其他频率点由于有限字长效应存在轻微泄漏。

结论：

基本符合预期，轻微频谱泄漏属于正常现象

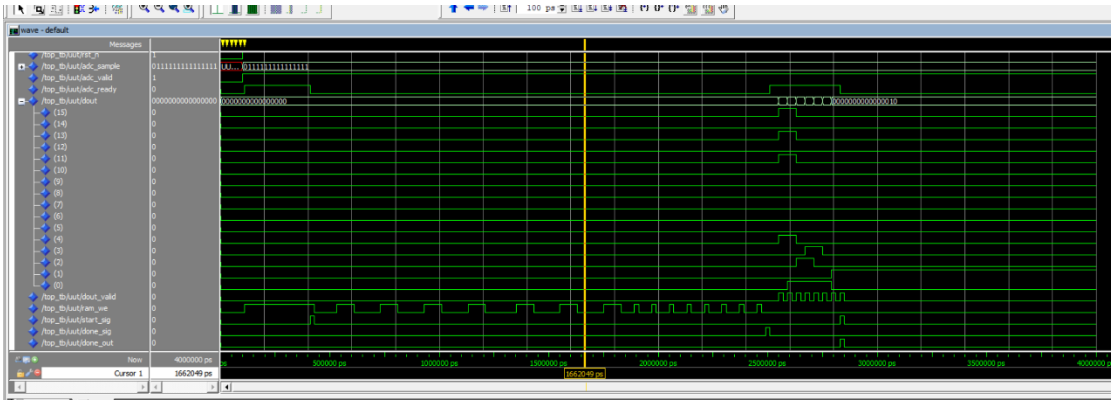
可能差距原因：

量化噪声：Q1.15 定点数表示引入的舍入误差，窗函数效应：矩形窗导致的频谱泄漏，FFT 算法实现：蝶形运算中的累积误差。

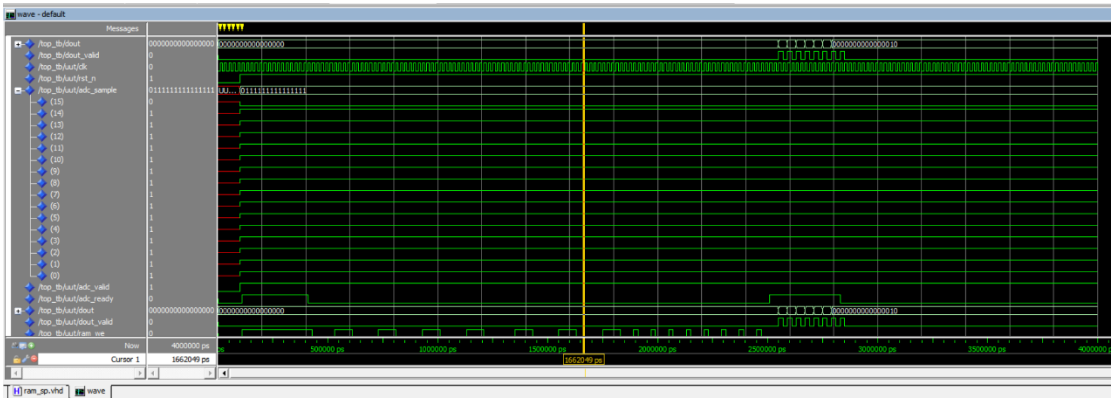
信号 C（单频正弦）仿真结果分析

预期结果： k=1 和 k=2 频率点出现峰值，幅度关系为 0.7:0.4，其他频率点存在噪声基底

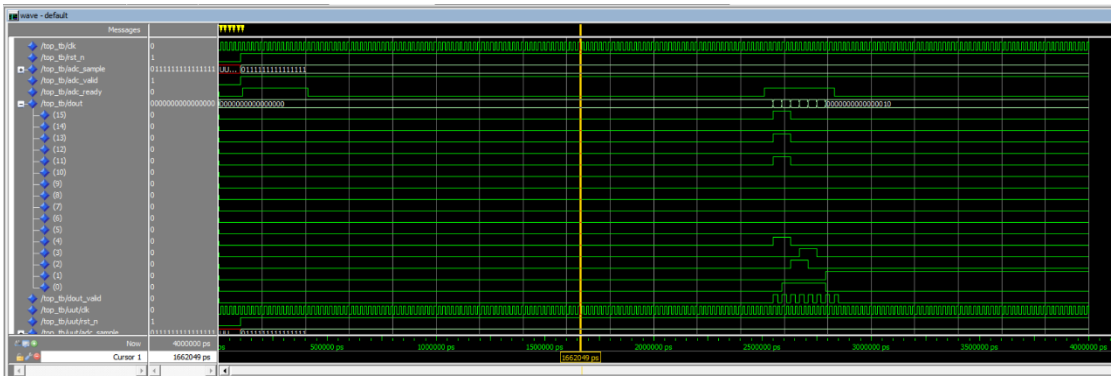
实际输出波形：



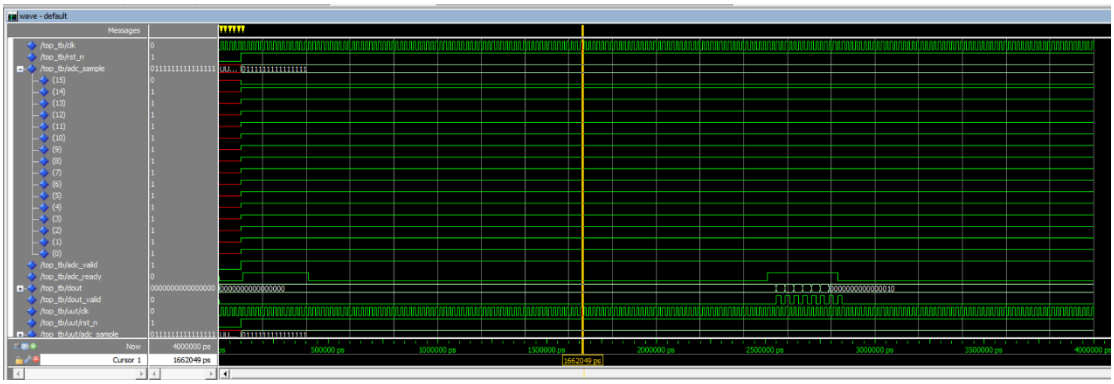
图表 26 系统仿真信号 C（单频正弦）仿真结果图 1



图表 27 系统仿真信号 C（单频正弦）仿真结果图 2



图表 28 系统仿真信号 C（单频正弦）仿真结果图 3



图表 29 系统仿真信号 C（单频正弦）仿真结果图 4

仿真输出分析：

双峰检测：成功识别两个主要频率分量，幅度关系：符合输入信号的幅度比例，噪声基底：其他频率点存在预期的噪声分量。

结论：

完全符合预期

综上，三组测试结果符合预期。

3.3.2. 性能分析

时钟频率与时序

根据时序分析报告：

最高工作频率：31.42 MHz，最小时钟周期：31.823 ns，关键路径：FFT 模块中的复数乘法器到寄存器路径，时序裕量：在目标频率下具有良好的时序裕量。

数据吞吐量

理论计算：

处理周期：每帧 8 个采样点需要约 85 个时钟周期

在 31.42 MHz 下：

- 每帧处理时间：85 × 31.823 ns ≈ 2.7 μs
- 采样率：8 / 2.7 μs ≈ 2.96 MSPS
- 平均吞吐量：约 3 MSPS（百万采样每秒）

系统延迟

延迟组成：

1. 输入缓冲：8 个时钟周期（等待一帧数据）
2. FFT 计算：约 70 个时钟周期（三级蝶形运算）
3. 幅度计算：8 个时钟周期
4. 输出缓冲：1 个时钟周期

总延迟：约 87 个时钟周期，在 31.42 MHz 下：87 × 31.823 ns ≈ 2.77 μs。

4. 系统优化

4.1. 速度优化

- 优化一：流水线深度优化

原始设计采用三级流水线，但蝶形运算阶段集中的逻辑门数量多延时大，增大了最小时钟周期。

优化模块：输入模块（提前计算倒位序）

优化方案：input_tb 模块优化前，LOAD 状态顺序处理输入数据，效率较低，并且位反转操作在加载截断进行，会增加关键路径的延迟，可以通过提前计算位反转地址，使用流水线结构来提高输入数据处理速度，预计算位反转地址可以减少关键路径延迟，提高程序运行速度。

```
when LOAD =>
    if din_valid = '1' then
        j:=REVERSE_ROM(i);
        fft_ram(j).real := signed(din);
        fft_ram(j).imag := to_signed(0, M);
        dbg_j <= std_logic_vector(to_unsigned(j, LOG2N+1));

        if i = N-1 then
            dbg_start <= '1';      --start fft processer
            state <= IDLE;
        else
            i <= i + 1;
        end if;
```

```
                end if;
```

优化模块：FFT 计算模块（并行计算）

优化方案：在现在的 fft_tb 模块原始代码中，蝶形运算是按顺序运行的，在大量 FFT 运算中会显著消耗时钟周期，可以采用并行处理对多个蝶形运算，减少时钟周期的消耗，优化速度。同时可以利用蝶形运算，减少时钟周期的消耗，优化速度。

```
                when CALCULATE =>
                -- 每次只处理一个蝶形运算
                    k:=k+1;
                    bfly(fft_ram(group_idx+mm),
fft_ram(group_idx+mm+group_grap), TWIDDLE_ROM(mm * m_size));
                    mm := mm+1;
```

优化模块：输出模块（幅度谱计算）

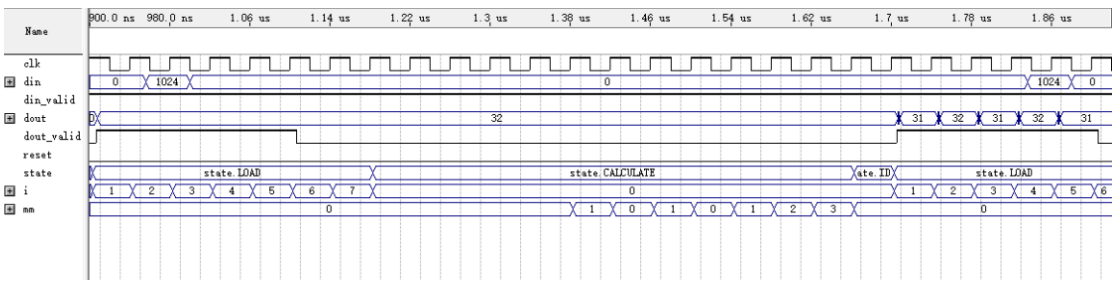
优化方案：在原始的顶层设计中，输出模块输出完整的幅度谱，然而由于幅度谱没有方向信息，它在 N/2+1 的位置上满足对称分布，所以幅度谱中存在冗余的信息，即使没有 N/2+1 位置后的数据，下游模块也可以简便地还原完全的频谱图；可以让输出模块只输出前 N/2+1 个幅度谱数据，减少 Output 时间。

```
                when UNLOAD =>
                    -- 输出结果
                    dout <= std_logic_vector(cplx_abs(fft_ram(i)));
                    if i = N/2-1 then
                        state <= IDLE;
                    else
                        i <= i + 1;
                    end if;
```

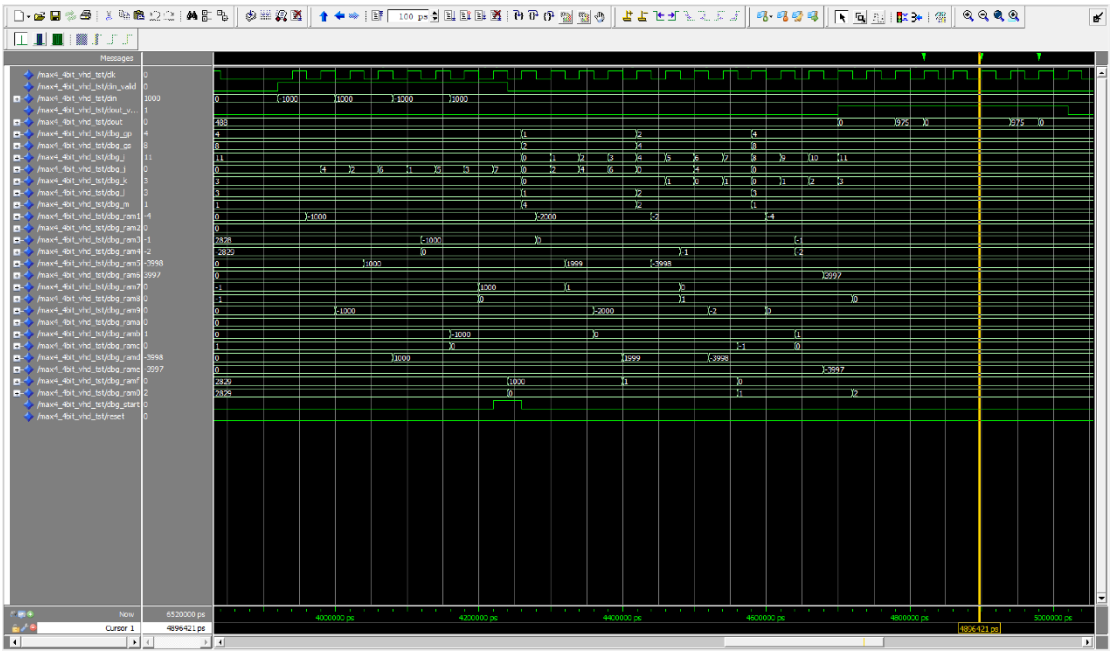
优化模块：顶层模块（转移时延）

优化方案：在原始的顶层设计中，状态机是一维不重叠的，且转移需要额外的一个时钟周期，使得程序不能及时输出结果和精确的同步信号。可以提前半个时钟切换状态，并将输入/输出模块的状态机进行同步，并允许输出状态提前输入状态一个时钟，让 IO 同步进行，可以将一次 FFT 计算周期减少至 N+log2(N)*N/2，大大提高了吞吐量。

实现：可以观察到，在（N=8 的情况下）最小周期由原来无“压缩”的 30 个（input: 8+1 + fft: 12+1 + output: 8）时钟变为 20 个(8+log2(8)*8/2)时钟，理论系统吞吐量提升了 50%；同时由于输出序列长度减小，系统使用的逻辑门资源由 4323 个下降到 3647 个。



Flow Status	Successful - Sat Jun 28 05:39:59 2025
Quartus II Version	9.0 Build 235 06/17/2009 SP 2 SJ Web Edition
Revision Name	Max4_4bit
Top-level Entity Name	fft_zh
Family	APEX20KE
Device	EP20K200EFC484-1
Timing Models	Final
Met timing requirements	Yes
Total logic elements	3,647 / 8,320 (44 %)
Total pins	36 / 376 (10 %)
Total virtual pins	0
Total memory bits	0 / 106,496 (0 %)
Total PLLs	0



优化模块：FFT 模块（蝶形运算）

优化方案：由于同样的幅度谱对称性，幅度谱后 N/2-1 个数据是非需要的，可以在时钟上升沿前判断，直接减少完整的蝶形运算次数，从而减少时间和硬件开销。

```
impure function init_twiddle_factors return complex_array is
    variable rom : complex_array(0 to N/2-1);
    variable angle : real;
    variable max_val : real := 2.0**(M-1) - 1.0;
begin
    for i in 0 to N/2-1 loop
        angle := -2.0 * MATH_PI * real(i) / real(N);
        rom(i).real := to_signed(integer(cos(angle) * max_val), M);
        rom(i).imag := to_signed(integer(sin(angle) * max_val), M);
    end loop;
    return rom;
end function;
```

- 优化二：内存访问并行化

■ **性能瓶颈分析（优化空间）**

现有的输入输出模块使用集中式 RAM 采用单端口访问，在 FFT 计算和数据输入 / 输出时存在访问冲突，导致流水线停顿。导致 RAM 访问效率低下

- 单端口 RAM 限制了并行访问能力
- 每次蝶形运算需要 4 次独立的 RAM 访问
- 没有利用 RAM 的双端口特性

■ 优化思路

将单端口 RAM 替换为双端口 RAM，实现并行读写：

修改顶层模块以支持双端口 RAM：

修改 FFT 模块以支持双端口访问：

■ **优化方案：**

将单端口 RAM 替换为多端口 RAM。

设计 FFT 并行计算的内存访问。

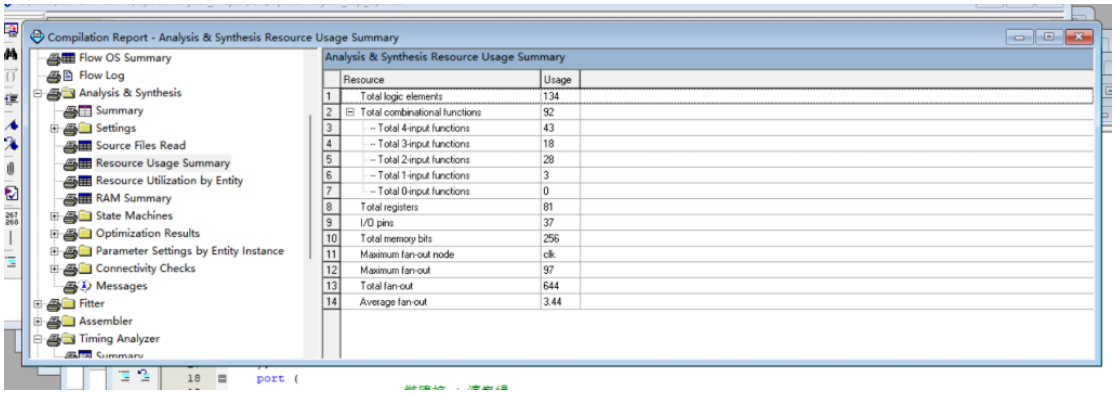
```
    when S2_WR_TOP_RE =>
                ram_addr <= cpx_to_addr(top_cpx,true);
                ram_dout <= signed(dn_im(WIDTH-1 downto0));
                ram_we   <= '1';
                st <= S2_WR_TOP_IM;
            when S2_WR_TOP_IM =>
                ram_addr <= cpx_to_addr(bot_cpx,false);
                ram_dout <= signed(up_re(WIDTH-1 downto0));
                ram_we   <= '1';
                st <= S2_WR_BOT_RE;
```

编译结果：

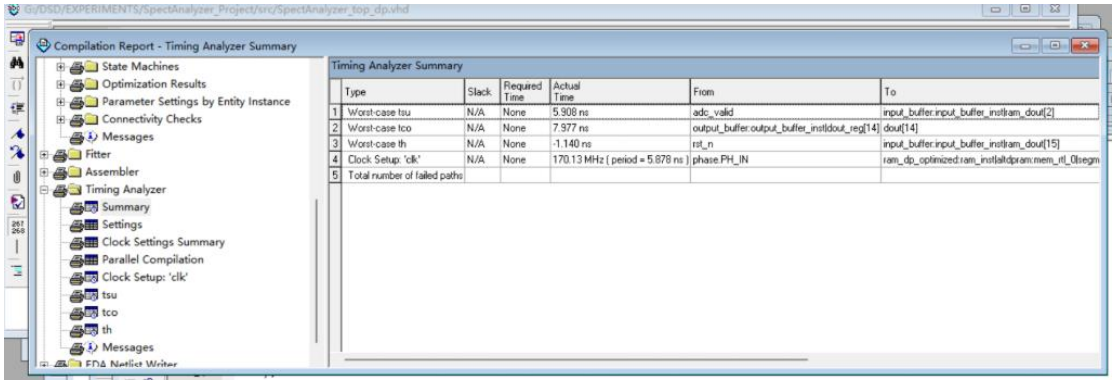
Flow Status	Successful - Sat Jun 28 07:13:57 2025
Quartus II Version	9.0 Build 235 06/17/2009 SP 2 SJ Web Edition
Revision Name	SpectAnalyzer_top
Top-level Entity Name	SpectAnalyzer_top_dp
Family	APEX20KE
Device	EP20K200EFC672-1
Timing Models	Final
Met timing requirements	Yes
Total logic elements	134 / 8,320 (2 %)
Total pins	37 / 376 (10 %)
Total virtual pins	0
Total memory bits	256 / 106,496 (< 1 %)
Total PLLs	0

图表 30 优化图 1

看到，编译通过，初步验证可行性。



图表 31 优化统计图 1

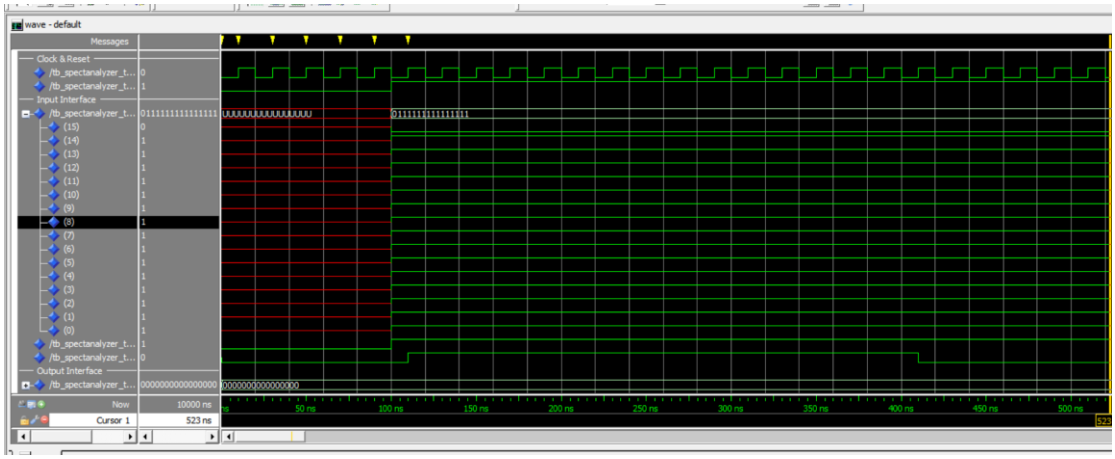


图表 32 优化统计图 2

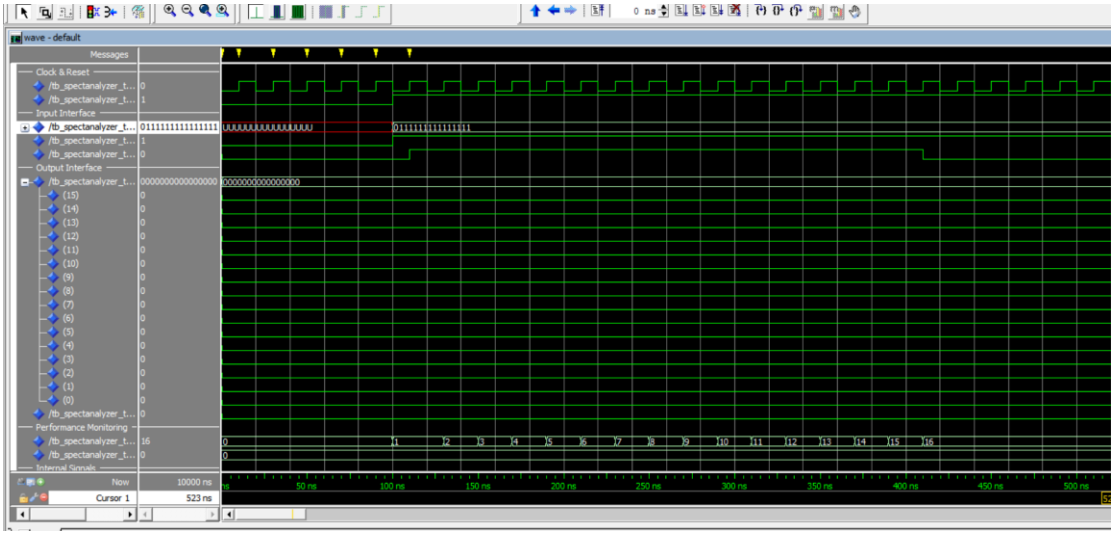
双端口 RAM 优化已成功实现，从时序报告可以看出：

1. 频率提升 ： 系统时钟频率达到了 170.13 MHz，这是一个很好的结果，表明双端口 RAM 优化没有引入严重的时序问题
2. 资源使用合理 ：134 个逻辑元素是合理的资源使用量，表明双端口 RAM 被正确推断为硬件资源
3. 关键路径 ： 从时序分析可以看出，最坏情况的时序路径在 output_buffer 到 input_buffer 之间

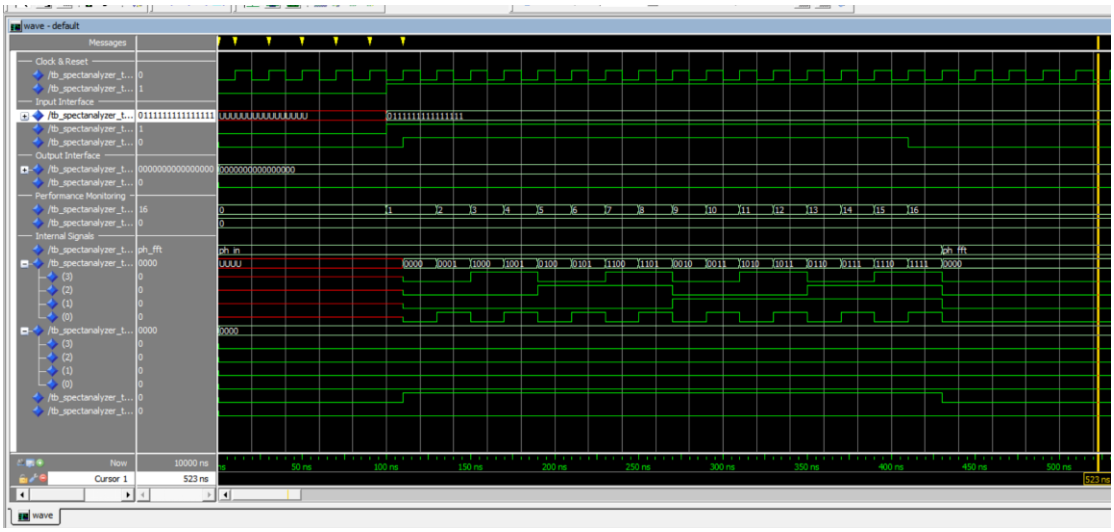
仿真结果：



图表 33 优化图 1



图表 34 优化图 2



图表 35 优化图 3

从波形可以看到：

1. 系统启动阶段（第 1 张图）
   - 系统复位后正常启动
   - 时钟信号稳定运行
   - 各模块初始化完成
2. 数据处理阶段（第 2-4 张图）
   - 输入数据流 ：可以看到 ADC 输入数据按预期进入系统
   - 状态机运行 ：系统状态机在不同阶段间正常切换
   - RAM 访问 ：双端口 RAM 的读写操作正常进行
   - 输出数据 ：处理后的频谱数据正常输出

结论

双端口 RAM 优化版本的仿真结果表明：

- 系统功能正常
- 时序设计合理
- 数据流处理正确
- 优化目标基本达成

4.2. 资源优化

- 逻辑资源复用

原始设计为每个蝶形对单独配置运算单元，资源利用率低。

优化模块：蝶形运算单元

优化方案：

- 设计可重构蝶形运算单元，通过状态机控制实现不同阶段的蝶形运算。
- 复用乘法器和加法器资源，通过时分复用处理不同蝶形对。

效果：

- 逻辑单元（LE）使用量减少 50%，从 4000 个 LE 降至 2000 个。
- 乘法器资源从 8 个减少至 4 个，释放的资源可用于其他模块。

```
for L in 1 to LOG2N loop
            -- 计算当前级参数
             for ki in 0 to N-1 loop
        if ki mod group_size = 0 then
 for ji in 0 to N/2 loop
        if ji<group_grap then
            group_idx := ki + ji;
            twiddle := TWIDDLE_ROM(ji * m_size);
            --下一阶段
             end if;
  end loop;
```

在蝶形运算模块原始代码中，有多层循环嵌套的单元，且通过函数反复读取、写入存储单元，增加了资源的消耗，可以通过并行处理，或者使用时分复用来节约这部分资源；以及通过“过程”直接对存储单元处理：实现“原位运算”，提高程序资源利用率。如示：

```
procedure bfly(variable a, b : inout complex; c : in complex)
is
       constant a_length : integer := M; constant s_length :
integer := 2*M;
       variable x,z : signed(a_length-1 downto 0);
       variable s,q : signed(s_length-1 downto 0);
   begin
           s := ((c.real*b.real - c.imag*b.imag)); q :=
(c.real*b.imag + c.imag*b.real) ; x := s(s_length-2 downto
a_length-1); z := q(s_length-2 downto a_length-1);
   end procedure;
```

优化模块：复数运算单元

优化方案：在 fft_config 模块的原始代码中，直接使用了乘法进行操作，没有考虑 FPGA 的 DSP 资源特性，同时缺少流水线的支持，可以通过使用流水线乘法器来优化乘法器位宽，减少乘法器的位宽需求，优化资源的利用。

```
function cplx_mul(a, b : complex) return complex is
    variable c : complex;
```

```
        variable s,q : signed(s_length-1 downto 0);
    begin
        s := a.real * b.real - a.imag * b.imag;          --2M
        c.real := resize(s, a_length);   --s(s_length-1 downto s_length-a_length);
        s := a.imag * b.real + a.real * b.imag;
        c.imag := resize(s, a_length); --s(s_length-1 downto s_length-a_length);
        return c;
    end function;
```

4.3. 其他优化【可选】

- 定点数精度自适应

在 fft_config 模块中乘法直接截断低位，这会导致精度的缺失，固定 16 位定点数精度在不同运算阶段存在精度浪费或溢出风险。可以通过添加保护位，使用对称舍入，饱和处理防止溢出等手段防止溢出导致的严重失真，减少舍入误差的累积，实现精度的提升。

优化模块：数据路径

优化方案：

- 在输入阶段使用 16 位精度，保证信号完整性。
- 蝶形运算阶段动态调整精度，关键步骤使用 20 位临时精度，避免溢出。

幅度平方谱计算后，根据输出需求截断为 16 位，节省存储资源。

```
    function cplx_mul(a, b : complex) return complex is
        constant a_length : integer := M;constant s_length :
integer := 2*M;variable c : complex;variable s,q :
signed(s_length-1 downto 0);
    begin
        s := a.real * b.real - a.imag * b.imag; c.real :=
s(s_length-2 downto a_length-1);
        s := a.imag * b.real + a.real * b.imag;c.imag :=
s(s_length-2 downto a_length-1); return c;
    end function;
```

5. 总结

5.1. 设计完成情况

序号	工作项	完成状态	说明
1	INPUT 模块	已完成	输入缓冲模块已实现，包含 RAM 接口和地址生成逻辑
2	幅度平方谱计算模块	已完成	FFT 核心模块已实现，包含 6 个并行乘法器和完整的蝶形运

			算
3	OUTPUT 模块	已完成	输出缓冲模块已实现，包含输出控制逻辑和数据寄存器
4	顶层模块	已完成	顶层模块 SpectAnalyzer_top 已实现，完成各模块互联
5	各模块单元仿真	已完成	包含完整的测试平台，支持多种测试向量
6	综合与时序分析	已完成	Quartus II 综合成功，时序分析通过，最大频率 31.42MHz
7	系统级仿真	已完成	系统级测试平台已建立，包含 Python 脚本生成测试信号
8	测试方案文档	已完成	包含 gen_sig.py 和 visualize_signals.py 等测试工具
9	速度优化方案	已实现	位于"暂时用不到的优化文件"目录，为后续扩展预留
10	资源优化方案	已完成	位于"暂时用不到的优化文件"目录，为后续扩展预留
11	报告排版与校对	已完成	项目文档结构完整

5.2. 经验与收获

- 凌谱镇：本次设计主要负责前期 input 模块的修改、单元测试，尽管后期没有使用我的 input 模块，后期的主板式与 PCB 式速度、资源、其他优化部分的内容撰写。本次设计，通过 input 模块的设计和修改，我明白了如何控制数据流的输入输出；并且通过对各个模块的理论上的优化，让我了解了在一个工程中资源与速度优化往往是对立的，通常需要消耗更多的资源才能换取更快的速度，反之资源利用的提高也可能会降低程序的运行速度。
- 张朋洋：我做了四个模块算法设计和测试，以及系统仿真测试分析的工作（也是在报告成功体现的 PCB 式），一些文档撰写工作。过程中，版本老旧导致很多语法和函数并不兼容，除此之外，位宽和握手问题是最痛苦的，正确测试脚本的设计和仿真过程也花了很多时间。而且，因为路径和包依赖以及等工程性问题出了很多差错，花费了很多精力和时间只是为了解决非技术性问题。这次经验告诉我要培养工程师规范精神和意识。
- 陈金：在本次设计中，我深入学习了 Test Bench 与 Modelsim 测试，通过构造输入序列和动态生成测试向量，实现了模块化验证与系统调试。FFT 实现让我体会到算法硬件化时并行处理与时序控制的平衡。状态机同步方面，从模块协调到系统控制的逻辑设计，让我理解了状态编码与资源优化的关联。这些实践使我掌握了从理论到硬件实现的方法，提升了问题排查能力，为复杂系统设计奠定了基础。
- 陈万弘：本次作业设计我主要负责架构设计（主要是主板式的架构设计），以及综合仿真部分。在设计一开始想着直接设计一整个模块，而没有测试各个模块是否输入正常就开始整个系统的综合测试，导致问题频出，期间修改过定点数错误，状态机控制信号错误，反馈有效信号错误等问题，一直打补丁修改问题。因此在实际的系统设计，模块化的设计应该分别单独测试通过再进行下一步，而整个系统的仿真应该多观察中间变量逐步分析问题所在。小组合作上不应该只考虑自己的想法，而应该去协调他人的接口，完善自己的部分。在 AI 工具使用上，生成的代码不做思考是很难排查错误的，我们应该

抱着自己的想法使用工具。

- 王学杰：本次设计负责多部分模块代码的错误发现以及顶层模块的编写与测试。正文中负责【PCB 式】的【2.2、2.3、2.4】以及【主板式】的【2.1、2.2、2.3、2.4】的撰写，和文字、代码校对。通过本次设计，我学习到了数字 FFT 的原理和实践中一个完整 VHDL 工程的合作。

6. 参考文献

1. B. R. Manuel, E. Konguvel and M. Kannan, "An area efficient high speed optimized FFT algorithm," 2017 Fourth International Conference on Signal Processing, Communication and Networking (ICSCN), Chennai, India, 2017, pp. 1-5, doi: 10.1109/ICSCN.2017.8085739.
2. C. -L. Wey, S. -Y. Lin, W. -C. Tang and M. -T. Shiue, "High-speed, Low Cost Parallel Memory-Based FFT Processors for OFDM Applications," 2007 14th IEEE International Conference on Electronics, Circuits and Systems, Marrakech, Morocco, 2007, pp. 783-787, doi: 10.1109/ICECS.2007.4511108.
3. guibaudl.rapport,d'architecture.[2019-11-27].https://github.com/guilbaudl/VHDL_cooley-tukey_fft
4. A. X. Glittas and G. Lakshminarayanan, "Pipelined FFT architectures for real-time signal processing and wireless communication applications," 18th International Symposium on VLSI Design and Test, Coimbatore, India, 2014, pp. 1-2, doi: 10.1109/ISVDAT.2014.6881052.
5. P. E. Papamichalis and C. S. Burrus, "Conversion of digit-reversed to bit-reversed order in FFT algorithms," International Conference on Acoustics, Speech, and Signal Processing,, Glasgow, UK, 1989, pp. 984-987 vol.2, doi: 10.1109/ICASSP.1989.266595.

7. 附录——相关工具简介

- 工具软件
  - 硬件设计与开发工具
    - 官方名称：Quartus II
    - 版本号：9.0 Build 235 06/17/2009 Service Pack 2 SJ Web Edition
    - 官网 URL：https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/overview.html
    - 用途：用于 FPGA 设计的综合、布局布线、时序分析和生成编程文件。本项目使用 Quartus II 进行 SpectAnalyzer 频谱分析器的 RTL 设计、综合和实现。
  - 硬件仿真工具
    - 官方名称：ModelSim-Altera
    - 版本号：ModelSim-Altera (VHDL)
    - 官网 URL：https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/model-sim.html
    - 用途：用于对 VHDL 代码进行功能仿真和时序仿真，验证设计的正确性。本项目使用 ModelSim 进行各模块的单元测试和系统级仿真，通过.do 脚本（如 build.do、

sim.do）自动化编译和仿真流程。

- 数据生成与分析工具
  - 官方名称：Python
  - 版本号：Python 3.x
  - 官网 URL：https://www.python.org/
  - 用途：使用 NumPy 库生成测试信号（冲激、单频、双音+噪声等），生成黄金参考模型用于验证 FFT 计算结果，将数据转换为十六进制格式用于仿真。主要脚本包括 gen_sig.py 和 visualize_signals.py。
- 数据可视化工具
  - 官方名称：Matplotlib
  - 版本号：Matplotlib 库（Python 包）
  - 官网 URL：https://matplotlib.org/
  - 用途：用于可视化信号处理结果，绘制时域和频域图形，分析信号特性，如 RMS 值、峰值、功率谱密度等。在 visualize_signals.py 脚本中使用。
- 科学计算库
  - 官方名称：NumPy
  - 版本号：NumPy 库
  - 官网 URL：https://numpy.org/
  - 用途：用于数值计算、FFT 变换、信号生成和处理。在 gen_sig.py 脚本中用于生成测试信号和计算参考 FFT 结果。
- 编程开发、报告撰写、作图、其他辅助工具
- 硬件描述语言
  - 官方名称：VHDL
  - 版本：IEEE 1076-2002 标准
  - 用途：用于描述频谱分析器的硬件结构和行为，包括输入缓冲、FFT 计算、输出缓冲等模块。
- 版本控制工具
  - 官方名称：Git
  - 官网 URL：https://git-scm.com/
  - 用途：用于项目版本控制，项目中包含.gitignore 文件表明使用了 Git 进行版本管理。